

Tugas 2

II4021 Kriptografi

Steganografi LSB pada Berkas Video AVI

Disusun oleh

Kelompok 8



Sendi Putra Alicia
(18223063)



Wijaksana Aptaluhung
(18223088)



Audy Alicia Renatha
Tirayoh (18223097)

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Pernyataan Tidak Melakukan Kecurangan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Sendi Putra Alicia



Wijaksana Aptaluhung



Audy Alicia Renatha Tirayoh

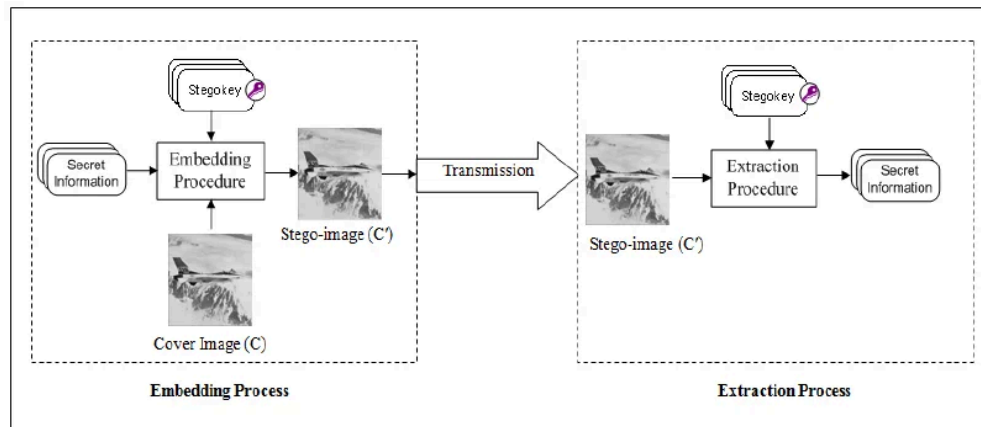
Daftar Isi

Daftar Isi	1
I. Teori Singkat	2
II. Perancangan dan Implementasi	8
III. Pengujian Program dan Analisis Hasil	19
IV. Kesimpulan Hasil Implementasi	38
V. Daftar Pustaka	39
VI. Lampiran	40

I. Teori Singkat

1. Steganografi

Steganografi adalah teknik menyembunyikan informasi rahasia di dalam suatu media pembawa (*cover*) sedemikian sehingga keberadaan pesan tidak mudah disadari oleh pihak lain [1]. Berbeda dari kriptografi yang mengubah isi pesan menjadi bentuk yang tidak bisa dipahami, steganografi justru menyembunyikan keberadaan pesan itu sendiri, pesan tetap dalam bentuk aslinya, namun tersimpan di dalam media yang tampak normal.



Gambar 1.1. *Steganography Embedding Extraction Diagram*

Dalam praktiknya, steganografi dan kriptografi sering digunakan bersamaan. Pesan dienkripsi terlebih dahulu menggunakan algoritma kriptografi, kemudian hasil enkripsi disisipkan ke dalam media *cover* menggunakan teknik steganografi. Kombinasi ini mempersulit steganalisis karena meskipun keberadaan pesan berhasil dideteksi, isinya tetap tidak bisa dibaca tanpa kunci yang benar.

Sebuah sistem steganografi yang baik harus memenuhi beberapa kriteria berikut [2]:

1. **Imperceptible**, perubahan pada media *cover* tidak dapat dipersepsi secara visual maupun pendengaran oleh indera manusia.
2. **Fidelity**, kualitas media *cover* tidak mengalami penurunan signifikan setelah penyisipan pesan.
3. **Recovery**, pesan yang telah disisipkan harus dapat diekstraksi kembali secara utuh.
4. **Capacity**, media *cover* mampu menampung pesan dengan ukuran yang memadai.

Media yang digunakan untuk menyembunyikan pesan dalam steganografi digital sangat beragam, di antaranya berkas gambar, audio, dan video. Media-media ini ideal digunakan karena memiliki **bit redundan – bit yang memberikan akurasi lebih dari yang dibutuhkan untuk merepresentasikan objek secara visual atau auditif** [2]. Perubahan pada

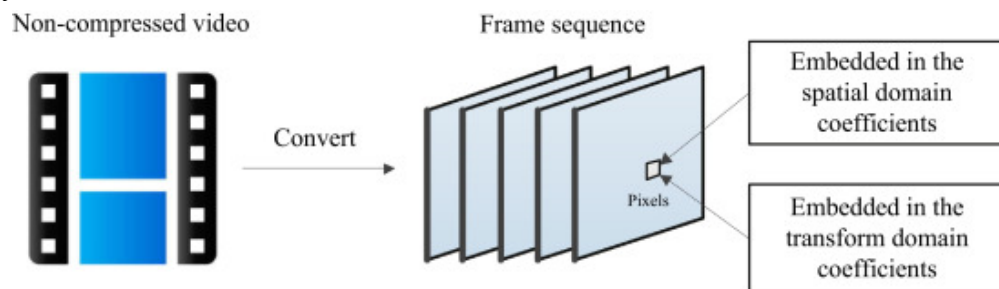
bit-bit redundan ini terlalu kecil untuk disadari, sehingga dapat dimanfaatkan untuk menyimpan informasi tambahan.

2. Format Video AVI

AVI (*Audio Video Interleave*) adalah format kontainer video yang dikembangkan oleh Microsoft. Format ini menyimpan data video dan audio secara bergantian dalam satu berkas, sehingga keduanya dapat diputar secara tersinkronisasi [3]. Berbeda dari format video modern seperti MP4 atau MKV yang umumnya menggunakan kompresi *lossy*, AVI dapat dikonfigurasi untuk menggunakan *codec lossless* atau bahkan tanpa kompresi sama sekali.

Sifat *lossless* inilah yang menjadikan AVI cocok digunakan sebagai media *cover* dalam steganografi berbasis LSB. Pada format *lossy*, proses kompresi dapat mengubah nilai piksel secara tidak terduga, sehingga bit-bit yang telah disisipkan berpotensi rusak atau berubah setelah video disimpan kembali [4]. Dengan menggunakan AVI tanpa kompresi, setiap perubahan kecil pada bit piksel akan tersimpan persis seperti yang ditulis.

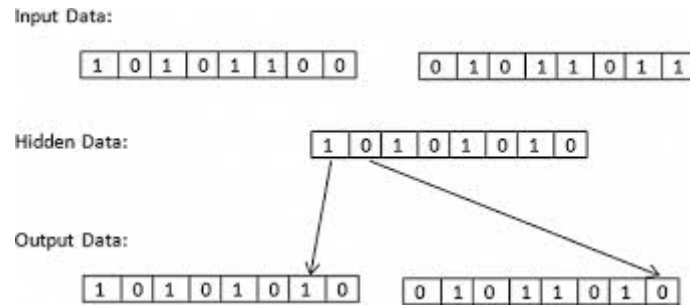
Setiap video pada dasarnya adalah kumpulan *frame*, yaitu gambar statis yang diputar secara berurutan dengan kecepatan tertentu yang disebut *frame rate*, biasanya dinyatakan dalam satuan *frames per second (fps)*. Setiap *frame* merupakan sebuah citra digital, sehingga teknik steganografi yang berlaku pada citra dapat diterapkan pula pada setiap *frame* video [3]. Hal ini menjadikan video memiliki kapasitas penyisipan yang jauh lebih besar dibandingkan citra tunggal, karena kapasitasnya bertambah seiring jumlah *frame*.



Gambar 1.2 Struktur Steganografi Video Frame

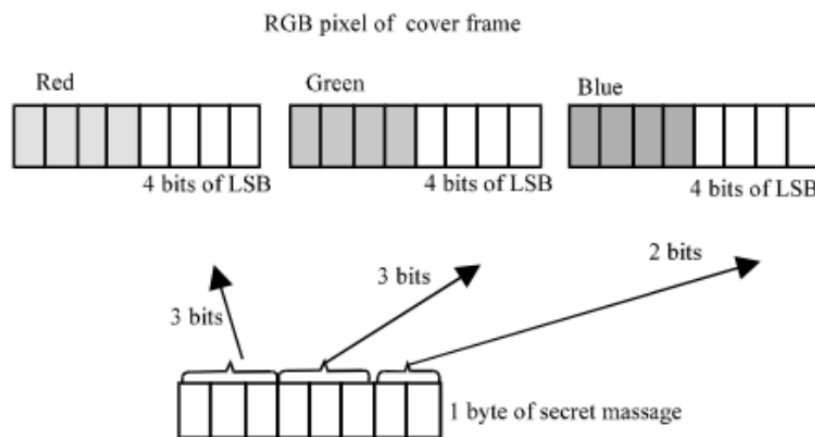
3. Metode LSB (*Least Significant Bit*)

LSB (*Least Significant Bit*) adalah salah satu metode steganografi yang paling sederhana dan paling banyak digunakan. Ide dasarnya adalah mengganti bit paling tidak signifikan dari setiap nilai piksel dengan bit pesan rahasia [2]. Karena bit paling kanan hanya berkontribusi sebesar 1 terhadap nilai piksel, perubahan yang dihasilkan sangat kecil dan tidak dapat dibedakan oleh mata manusia.



Gambar 1.3 Penggantian Bit dengan Pesan Rahasia

Setiap piksel pada citra berwarna terdiri dari tiga *channel* warna: *Red* (R), *Green* (G), dan *Blue* (B), masing-masing bernilai 0-255 dan direpresentasikan dalam 8 bit. Metode LSB memanfaatkan bit-bit paling bawah dari setiap *channel* untuk menyimpan bit pesan. Skema penyisipan dapat bervariasi tergantung berapa bit per *channel* yang digunakan, misalnya skema 3-3-2 yang menyisipkan 3 bit pada R, 3 bit pada G, dan 2 bit pada B sehingga total 8 bit (1 byte) pesan tersimpan per piksel [5].



Gambar 1.4 Contoh Skema LSB 3-3-2. Sumber: [Nados, W. L., & Rasheed, N. A. \(2019\)](#)

Secara umum terdapat dua mode penyisipan LSB [2]:

1. **Sequential**, bit pesan disisipkan secara berurutan mulai dari piksel pertama, bergerak dari kiri ke kanan dan atas ke bawah pada setiap *frame*, lalu dilanjutkan ke *frame* berikutnya.
2. **Random**, urutan piksel yang digunakan untuk penyisipan diacak menggunakan PRNG (*Pseudo Random Number Generator*) dengan seed yang berasal dari *stego-key*. Penerima pesan dapat merekonstruksi urutan yang sama dengan menggunakan *seed* yang identik saat proses ekstraksi [2].

Mode *random* umumnya dianggap lebih aman karena distribusi bit pesan yang tersebar acak lebih sulit dianalisis dibandingkan distribusi sekuensial.

4. Pengukuran Kualitas

Dalam steganografi digital, kualitas media hasil penyisipan (*stego-object*) merupakan aspek penting yang harus diperhatikan. Salah satu tujuan utama steganografi adalah menjaga agar perubahan pada media pembawa (*cover*) tidak dapat dideteksi secara visual, atau disebut sebagai *imperceptibility*.

Untuk mengukur kualitas *stego-video* dibandingkan dengan video asli, digunakan beberapa metrik kuantitatif, di antaranya **Peak Signal-to-Noise Ratio (PSNR)** dan **Mean Squared Error (MSE)**. Kedua metrik ini umum digunakan dalam pengolahan citra digital dan steganografi karena mampu merepresentasikan tingkat distorsi akibat proses penyisipan data.

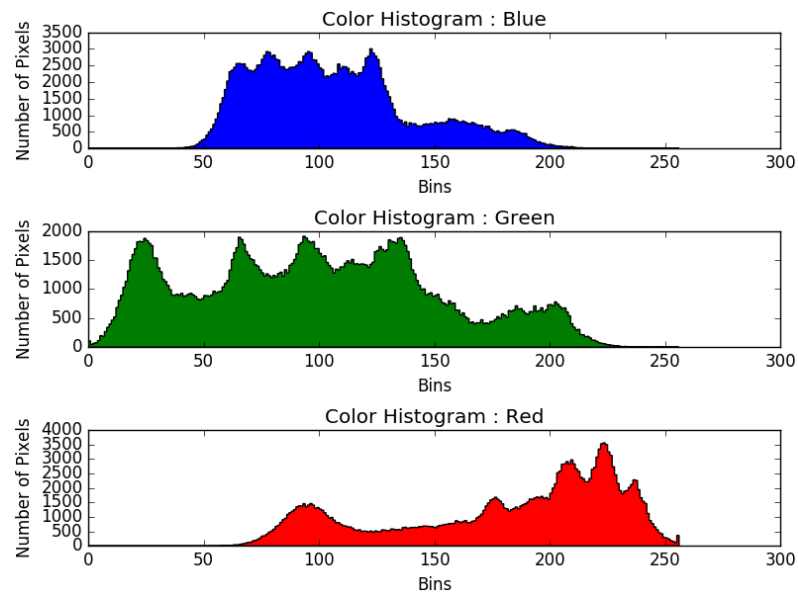
PSNR digunakan untuk menghitung rasio antara daya maksimum sinyal terhadap derau yang merusak. Nilai PSNR yang tinggi menunjukkan kualitas *stego-video* yang baik karena *noise* yang dihasilkan relatif kecil. Sebaliknya, nilai PSNR yang rendah mengindikasikan adanya distorsi yang lebih besar.

$$PSNR = 10 \times \log_{10} \left(\frac{MAX_I^2}{MSE} \right)$$

Sedangkan MSE digunakan untuk menghitung rata-rata selisih kuadrat antara piksel *video cover* (C) dan *stego-video* (S). MSE digunakan untuk mengetahui seberapa besar perubahan yang terjadi akibat proses *embedding*. Nilai MSE yang rendah menunjukkan bahwa perbedaan antara *cover* dan *stego* sangat kecil, sehingga kualitas visual tetap terjaga.

$$MSE = \frac{1}{M \times N} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} [C(i, j) - S(i, j)]^2$$

Selain menggunakan MSE dan PSNR, analisis kualitas juga dapat dilakukan dengan membandingkan **histogram warna** antara media asli dan *stego*. Histogram menggambarkan distribusi intensitas piksel untuk setiap *channel* warna (*Red, Green, Blue*). Jika histogram antara *cover* dan *stego* tidak menunjukkan perbedaan yang signifikan, maka dapat disimpulkan bahwa proses penyisipan tidak merusak karakteristik visual media secara mencolok.

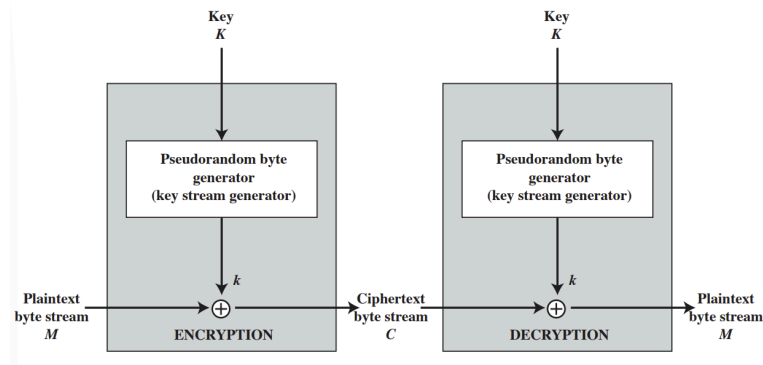


Gambar 1.5 Contoh Histogram RGB

5. *Stream Cipher*

Secara garis besar, *stream cipher* merupakan suatu metode enkripsi yang berjalan secara simetris, dilakukan satu-per-satu per bit dengan menggabungkan *plaintext* dengan suatu *keystream* yang diproduksi secara acak. Metode ini mengacak *plaintext* untuk membuat dan *generate* suatu *ciphertext* yang tidak mudah dipahami oleh pengguna yang tak terotorisasi atau bahkan pihak dengan niatan yang jahat. Karena sifatnya yang mengenkripsi data bit-per-bit, proses enkripsi cenderung cepat. Metode ini memerlukan tiga komponen utama: pesan yang ingin disembunyikan, *key*, serta algoritma enkripsi yang digunakan.

Alur kerja *stream cipher* cukup sederhana. Ini dimulai dari *plaintext* atau pesan asli, yaitu data yang ingin dienkripsi. Pengguna juga memerlukan suatu *key* awal. *Key* awal ini akan diberikan suatu algoritma tertentu sehingga akan dibuat suatu *keystream* yang sesuai dengan ukuran dari pesan yang ingin dienkripsi. Lalu, setiap bit dari *plaintext* akan diberi operasi XOR dengan tiap bit *keystream* sehingga menghasilkan suatu *ciphertext*. Untuk melakukan dekripsi, penerima cukup membuat *keystream* yang sama dengan menggunakan *key* dan algoritma yang sama dengan proses enkripsi, lalu dilakukan operasi XOR antara *ciphertext* dengan *keystream*, menghasilkan *plaintext* awal.



Gambar 1.6 Diagram Cara Kerja dari *Stream Cipher*

Kelebihan utama dari *stream cipher* adalah kecepatan enkripsi/dekripsinya yang hampir instan sehingga sederhana, ringan, dan *real-time friendly*. Namun, hal ini juga berarti bahwa keamanan cukup lemah dan rentan apabila *keystream* bocor sehingga *keystream* dan *key* harus benar-benar aman dan dirahasiakan.

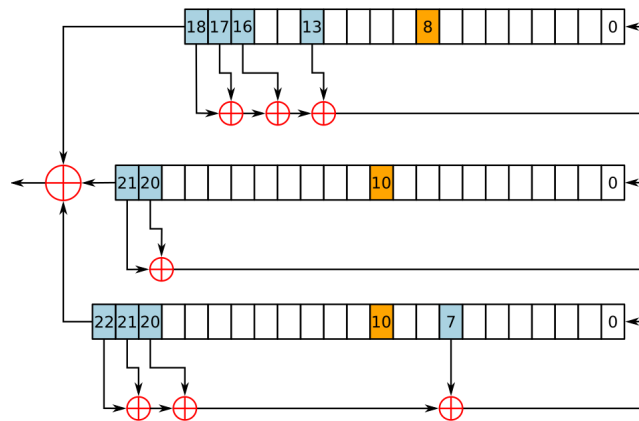
6. A5/1

A5/1 merupakan suatu algoritma enkripsi aliran dalam *stream cipher* yang awalnya dirancang untuk keamanan komunikasi via suara pada standar GSM (Global System for Mobile Communications). Dalam *stream cipher*, algoritma ini berperan sebagai penghasil *keystream* yang kelak akan digunakan untuk mengenkripsi tiap-tiap bit atau *frame* data awal sehingga pesan dapat dirahasiakan. Algoritma ini termasuk kategori *stream cipher* berbasis Linear Feedback Shift Register (LFSR) yang dikombinasikan dengan *clock-control mechanism* yang nonlinear agar kompleksitas *keystream* meningkat.

Algoritma ini didasari oleh tiga *register* dengan panjang 19, 22, dan 23 bit (totalnya 64 bit) yang biasanya disebut R_1 , R_2 , dan R_3 . Tiap *register* dilakukan *clock* atau pergeseran bila bit pada posisi tertentu termasuk bit yang merupakan mayoritas. Secara umum, algoritma di awal diberikan kunci 64-bit dan nomor *frame* publik sebesar 22 bit. Lalu, kunci dimasukkan ke dalam *register* dengan operasi XOR terhadap *register* tertentu sebanyak 64 kali. Lalu, nomor *frame* juga dimuat ke *register* sebanyak 22 siklus. Setelah itu, sistem akan dijalankan sebanyak 100 siklus dengan tidak memperhatikan *keystream* hasilnya, bertujuan agar *keystream* lebih sulit diprediksi. Setelah *warm-up* selesai akan dihasilkan *keystream* sebesar 228-bit yang kemudian di-XOR dengan bit *plaintext* untuk menghasilkan *ciphertext*. Dekripsi dilakukan dengan menghasilkan *ciphertext* dengan algoritma yang sama, lalu melakukan operasi XOR dengan *ciphertext* sehingga *plaintext* pun dihasilkan.

Clocking bit dari tiap-tiap *register* secara berurutan adalah 8, 10, dan 10. Ketiga *clocking bit* ini akan digunakan untuk memilih nilai mayoritas dari bit ini, menentukan dilakukan atau tidaknya *clocking*. Pada tiap-tiap *register*,

juga dipilih bit-bit yang akan di-XOR dan menjadi *input* baru untuk *register* tersebut ketika dilakukan *clocking*. R1 menggunakan bit ke-13, 16, 17, dan 18, R2 menggunakan bit ke-20 dan 21, dan R3 menggunakan bit ke-7, 20, 21, dan 22. Angka-angka ini digunakan sebagai standar dalam diimplementasikannya algoritma ini.



Gambar 1.7 Diagram *Register* dan Bit Spesial dari Algoritma A5/1

II. Perancangan dan Implementasi

1. *Tech Stack* dan Arsitektur Program

Program ini dibentuk menggunakan bahasa pemrograman Python dengan arsitektur yang terdiri atas dua bagian: antarmuka dan logika utama dari program. Untuk antarmuka atau GUI, digunakan *library* CustomTkinter yang dapat membuat GUI *modern* yang menarik sehingga program bisa lebih *accessible*. Sedang untuk logika utama, digunakan OpenCV untuk membaca dan menulis *frame* video, serta digunakan juga NumPy untuk manipulasi piksel dalam *frame*.

Struktur kode sendiri dibagi menjadi dua *package*: `gui/` untuk antarmuka dan `core/` untuk logika utama. Bagian GUI terdiri atas `main.py` yang menjadi *entry point* untuk `embed_tab.py` dan `extract_tab.py` yang mengimplementasikan *tab* untuk *embedding* dan *extraction*. Serta `metrics.py` untuk mengukur kualitas video (PSNR/MSE dan Histogram). *Tab* berhubungan dengan bagian logika utama dengan `stego_service.py` yang menangani validasi *input*, menjalankan proses enkripsi dan steganografi, serta melempar hasil kembali ke GUI.

Sedangkan lapisan *core* berisi modul yang memiliki fungsinya masing-masing: `video_io.py` yang menangani *input* dan *output* video, `A5_1.py` yang menangani proses enkripsi, `metadata.py` yang menangani pembuatan *header* yang menyimpan informasi, `embedding.py` yang menangani penyisipan bit ke piksel, serta `mp4_core.py` sebagai lapisan keamanan ganda (replikasi/redundansi) untuk penyisipan spesifik yang tahan terhadap kompresi format MP4 *Lossy*. Terakhir, modul `stego.py` menyatukan seluruh proses

orkestrasi steganografi tersebut. Pemisahan ini dilakukan agar sistem lebih modular, mudah dipelihara, dan tangguh untuk diuji sendiri-sendiri."

2. Pemrosesan Video AVI

Untuk membaca video masukan (cover video) berekstensi AVI, program memanfaatkan library OpenCV melalui fungsi `cv2.VideoCapture`.

```
def read_video(video_path):  
    path = Path(video_path)  
    if not path.exists():  
        raise FileNotFoundError(f"File tidak ditemukan: {video_path}")  
  
    cap = cv2.VideoCapture(str(path))  
    if not cap.isOpened():  
        raise ValueError(f"Tidak bisa membuka video: {video_path}")  
  
    info = {  
        'width': int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),  
        'height': int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)),  
        'fps': cap.get(cv2.CAP_PROP_FPS),  
        'frame_count': int(cap.get(cv2.CAP_PROP_FRAME_COUNT)),  
    }  
    frames = []  
    while True:  
        ret, frame = cap.read()  
        if not ret:  
            break  
        frames.append(frame)  
  
    cap.release()  
    info['frame_count'] = len(frames)  
    return frames, info
```

Lewat fungsi ini, video diekstrak menjadi sekumpulan *frame* yang berurutan. Setiap *frame* yang terbaca tersebut direpresentasikan sebagai array tiga dimensi milik NumPy berukuran (*height*, *width*, 3) bertipe data `uint8` (bernilai 0-255). Angka 3 pada dimensi terakhir menunjukkan tiga saluran warna yang digunakan, yakni *Blue*, *Green*, dan *Red* (format BGR khas bawaan OpenCV).

Selagi mengekstrak *frame*, program juga menyimpan metadata asli berupa informasi lebar (*width*), tinggi (*height*), fps, dan total *frame* untuk dijadikan referensi nanti. Saat proses *embedding* bit rahasia selesai, array NumPy yang telah dimodifikasi akan dirakit kembali menjadi *stego-video*. Pada tahap penulisan ini, program menggunakan fungsi `cv2.VideoWriter` beserta codec RGBA.

```
def write_video(output_path, frames, info):
```

```

# cek format file, kalau mp4 pake codec mp4v
if str(output_path).lower().endswith('.mp4'):
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
else:
    fourcc = cv2.VideoWriter_fourcc(*LOSSLESS_CODEC)

writer = cv2.VideoWriter(output_path, fourcc, info['fps'],
(info['width'], info['height']))

if not writer.isOpened() and not
str(output_path).lower().endswith('.mp4'):
    # fallback buat .avi kalau RGBA gagal
    fourcc = cv2.VideoWriter_fourcc(*'png ')
    writer = cv2.VideoWriter(output_path, fourcc, info['fps'],
(info['width'], info['height']))

if not writer.isOpened():
    raise ValueError("Gagal membuat video output, codec tidak
support")

for frame in frames:
    writer.write(frame)

writer.release()

```

Penggunaan *codec* ini sangat penting karena sifatnya yang murni lossless (tidak memampatkan data secara merusak). Dengan begitu, modifikasi halus level bit yang sudah dipasangkan pada piksel tidak akan corrupt atau hilang saat video disimpan. Sebagai langkah antisipasi jika sistem operasi pengguna tidak mendukung *codec* dasar RGBA, program telah disiapkan untuk langsung memicu fallback otomatis ke *codec* format png yang sama-sama *lossless*.

3. Metode Enkripsi & Dekripsi A5/1 yang Diimplementasikan

Program akan menerima pesan (teks atau *file*) beserta pilihan pengguna: apakah pengguna ingin mengenkripsikan pesan atau tidak? Apabila pengguna memilih iya, pengguna harus melakukan *input* berupa *key* yang akan digunakan dalam proses enkripsi. *Key* yang dimasukkan harus berupa *integer*, yang kelak akan menjadi *key* dalam bentuk bit dengan ukuran 64-bit.

Lalu, akan dibaca *input* dan diubah menjadi *raw bytes*. Apabila dipilih untuk dilakukan enkripsi, *raw bytes* akan dikonversi menjadi urutan bit, dengan penambahan *padding* agar kelipatannya 228. Blok akan dienkripsi secara terpisah per 228-bit. Tiap-tiap blok akan diberi nomor dari 0 hingga menaik terus. Lalu, akan dimasukkan ke mesin A5/1 *key* dan *frame number* yang menghasilkan suatu *keystream* unik. *Keystream* ini akan diberi operasi

XOR dengan blok *plaintext* yang hasilnya dikonversi ke *bytes* sebagai suatu *payload* yang terenkripsi.

```
def encrypt_logic(self, raw_bytes, key_int, start_fn=0):

    bit_list = self.bytes_to_bits(raw_bytes)

    while len(bit_list) % 228 != 0:

        bit_list.append(0)

    encrypted_bits = []

    current_fn = start_fn

    for i in range(0, len(bit_list), 228):

        blok = bit_list[i:i+228]

        self.engine.initialize(key_int, current_fn)

        keystream = self.engine.get_keystream_block()

        cipher_blok = [b ^ k for b, k in zip(blok, keystream)]

        encrypted_bits.extend(cipher_blok)

        current_fn += 1

    payload_encrypted = self.bits_to_bytes(encrypted_bits)

    return payload_encrypted
```

Proses dekripsi dilakukan dengan cara yang mirip. Ketika pengguna ingin melakukan ekstrak pesan, akan ada pilihan apakah pengguna mengenkripsikan pesan pada *file* yang ingin diekstrak. Bila iya, pengguna harus memasukkan *key* yang digunakan dalam proses *embedding*. Lalu, *payload* akan diekstrak dari *frame* video. Karena sifat XOR yang *self-inverse*, akan dilakukan proses serupa yang menghasilkan blok beserta *keystream* yang sama dan dilakukan XOR sehingga menghasilkan *plaintext*. *Padding* yang diberikan akan dipotong sehingga hasil sama persis dengan pesan asli. *Bytes* hasil dekripsi ini akan dikembalikan menjadi *text* atau *file* awal.

```
def decrypt_logic(self, encrypted_payload, key_int, original_size,
start_fn=0):

    decrypted = self.encrypt_logic(encrypted_payload, key_int,
start_fn)

    return decrypted[:original_size]
```

4. Metode LSB yang Diimplementasikan

Program mendukung tiga konfigurasi skema LSB yang dapat dipilih saat program dijalankan:

Tabel 2.1 Skema LSB.

Skema	R	G	B	Bit/piksel
1-1-1	1	1	1	3
3-3-2	3	3	2	8
4-4-4	4	4	4	12

Pada proses *embedding*, setiap bit pesan disisipkan ke LSB *channel* R, G, B sesuai skema yang dipilih menggunakan operasi *bitwise*. Untuk mengganti bit ke- i pada nilai *channel* c dengan bit pesan b :

$$c = (c \& (0xFF \wedge (1 \ll i))) \mid (b \ll i)$$

```
def embed_bits_to_frame(frame, bits, scheme, start_idx=0):
    r_bits, g_bits, b_bits = LSB_SCHEMES[scheme]
    result = frame.copy()
    height, width = frame.shape[:2]

    bit_idx = 0
    total_bits = len(bits)
    pixel_idx = 0

    for y in range(height):
        for x in range(width):
            if pixel_idx < start_idx:
                pixel_idx += 1
                continue
            if bit_idx >= total_bits:
                return result, bit_idx

            b, g, r = int(result[y, x, 0]), int(result[y, x, 1]),
int(result[y, x, 2])

            # embed ke R
            for i in range(r_bits):
                if bit_idx >= total_bits:
                    break
                r = (r & (0xFF ^ (1 << i))) | (bits[bit_idx] << i)
                bit_idx += 1

            # embed ke G
            for i in range(g_bits):
                if bit_idx >= total_bits:
                    break
                g = (g & (0xFF ^ (1 << i))) | (bits[bit_idx] << i)
```

```

        bit_idx += 1

    # embed ke B
    for i in range(b_bits):
        if bit_idx >= total_bits:
            break
        b = (b & (0xFF ^ (1 << i))) | (bits[bit_idx] << i)
        bit_idx += 1

    result[y, x] = [b, g, r]
    pixel_idx += 1

return result, bit_idx

```

Operasi ini memastikan hanya bit yang ditarget yang berubah, sedangkan bit lainnya tetap. Proses ekstraksi adalah kebalikannya, dimana bit ke- i diambil dari *channel* dengan operasi $(c \gg i) \& 1$.

```

def extract_bits_from_frame(frame, num_bits, scheme, start_idx=0):

    r_bits, g_bits, b_bits = LSB_SCHEMES[scheme]

    height, width = frame.shape[:2]

    bits = []

    pixel_idx = 0

    for y in range(height):

        for x in range(width):

            if pixel_idx < start_idx:

                pixel_idx += 1

                continue

            if len(bits) >= num_bits:

                return bits

            b, g, r = frame[y, x]

            for i in range(r_bits):

```

```

        if len(bits) >= num_bits:

            break

        bits.append((r >> i) & 1)

    for i in range(g_bits):

        if len(bits) >= num_bits:

            break

        bits.append((g >> i) & 1)

    for i in range(b_bits):

        if len(bits) >= num_bits:

            break

        bits.append((b >> i) & 1)

    pixel_idx += 1

return bits

```

5. Pemilihan Frame dan Urutan Piksel

Pemilihan *frame*: program menyisipkan pesan ke seluruh *frame* video secara lintas *frame*. Daftar posisi piksel dibangun dengan iterasi (*frame_index*, *y*, *x*) untuk setiap piksel di setiap *frame*. Apabila satu *frame* telah terisi penuh, penyisipan otomatis dilanjutkan ke *frame* berikutnya tanpa perlu penanganan khusus.

Mode *sequential*: posisi piksel diproses sesuai urutan natural, dari kiri ke kanan, atas ke bawah, *frame* pertama hingga terakhir.

Mode *random*: seluruh posisi piksel dituliskan sebagai indeks dalam *array* NumPy, diacak dengan `np.random.default_rng(seed).shuffle()`. *Stego-key* berupa *string* yang di-input *user*, lalu di-hash menjadi integer menggunakan fungsi `hash()` dari Python, kemudian digunakan sebagai *seed* untuk NumPy *random generator*.

```

def get_pixel_order(total_pixels, mode, seed=None):

```

```

indices = np.arange(total_pixels, dtype=np.int64)

if mode == 'random':

    if seed is None:

        raise ValueError("random mode butuh seed")

    # convert seed ke integer hash buat NumPy RNG

    seed_int = int(hash(seed)) % (2**63)

    rng = np.random.default_rng(seed=seed_int)

    rng.shuffle(indices)

return indices

```

Pendekatan ini dipilih karena fungsi untuk *shuffle* yang digunakan bersifat deterministik, yang membuat *seed* yang sama menghasilkan urutan acak yang identik sehingga proses ekstraksi bisa menghasilkan urutan yang sama melalui seed yang sama. Operasi ini juga performanya yang sangat baik dan cepat.

6. Penyimpanan Metadata/Header

Pada proses ekstraksi, program terlebih dahulu membaca *header* secara mandiri di awal *bitstream* (maksimal 520 byte) sebelum mengambil *payload*. Hal ini penting karena ukuran *payload* bersifat dinamis, sehingga tanpa membaca *header* terlebih dahulu, program tidak dapat menentukan jumlah bit yang harus diekstraksi secara tepat dan berisiko membaca data kurang atau berlebih.

Header disisipkan di awal *bitstream* dan berfungsi menyimpan metadata yang dibutuhkan untuk ekstraksi, Format header yang digunakan adalah sebagai berikut:

Field	Ukuran	Keterangan
Ukuran <i>payload</i>	4 byte	Panjang <i>payload</i> dalam byte sebelum diberi <i>padding</i>
Jenis Pesan	1 byte	0 berarti pesan berupa teks, 1 berarti pesan berupa <i>file</i>
Flag Enkripsi	1 byte	0 berarti tidak dilakukan enkripsi, 1 berarti dilakukan enkripsi

<i>Flag Metode Penyisipan</i>	1 <i>byte</i>	0 berarti penyisipan dilakukan sekuensial, 1 berarti penyisipan dilakukan <i>random</i> sesuai dengan <i>stego-key</i>
Skema LSB	1 <i>byte</i>	0 berarti menggunakan skema 3-3-2, 1 berarti menggunakan skema 4-2-2, 2 berarti menggunakan skema 2-2-2, dan 99 berarti menggunakan skema mp4-robust (khusus <i>lossy</i> mp4)
Panjang <i>Extension</i>	1 <i>byte</i>	Menyimpan urutan jumlah byte dari teks ekstensi file
<i>Extension</i>	N <i>byte</i> (dinamis, sesuai dengan <i>extension</i>)	Teks nama ekstensi file sasaran (jika pesan berupa file)
Panjang Nama <i>File</i> Asli	1 <i>byte</i>	Ukuran dari nama <i>file</i> asli (apabila pesan yang disisipkan berupa <i>file</i>)
Nama <i>File</i> Asli	M <i>byte</i> (dinamis, sesuai dengan panjang nama)	Nama <i>file</i> asli untuk pembentukan kembali <i>file</i> dari pesan (apabila pesan yang disisipkan berupa <i>file</i>)

```
def build_header(info):

    ext = info.get('extension', '').encode()

    nama = info.get('nama_file', '').encode()

    message_type = _resolve_message_type(info)

    mode = info.get('mode', 'sequential')

    if mode not in MODE_TO_CODE:

        raise ValueError(f"Mode sisip tidak dikenal: {mode}")
```

```

    scheme = info.get('scheme', '3-3-2')

    if scheme not in SCHEME_TO_CODE:

        raise ValueError(f"Skema LSB tidak dikenal untuk metadata:
{scheme}")

    h = struct.pack('>I', info['ukuran'])

    h += bytes([message_type])

    h += bytes([1 if info['is_encrypted'] else 0])

    h += bytes([MODE_TO_CODE[mode]])

    h += bytes([SCHEME_TO_CODE[scheme]])

    h += bytes([len(ext)]) + ext

    h += bytes([len(nama)]) + nama

    return h

def parse_header(data):

    i = 0

    ukuran = struct.unpack('>I', data[i:i+4])[0]

    i += 4

    message_type = data[i]; i += 1

    if message_type not in (0, 1):

        raise ValueError(f"Jenis pesan tidak valid di header:
{message_type}")

    is_encrypted = bool(data[i]); i += 1

    mode_code = data[i]; i += 1

```

```
scheme_code = data[i]; i += 1

if mode_code not in CODE_TO_MODE:

    raise ValueError(f"Mode code tidak dikenal: {mode_code}")

if scheme_code not in CODE_TO_SCHEME:

    raise ValueError(f"Scheme code tidak dikenal: {scheme_code}")

ext_len = data[i]; i += 1

extension = data[i:i+ext_len].decode(); i += ext_len

nama_len = data[i]; i += 1

nama_file = data[i:i+nama_len].decode(); i += nama_len

return {

    'ukuran': ukuran,

    'message_type': message_type,

    'is_text': message_type == 0,

    'is_file': message_type == 1,

    'is_encrypted': is_encrypted,

    'mode': CODE_TO_MODE[mode_code],

    'scheme': CODE_TO_SCHEME[scheme_code],

    'extension': extension,

    'nama_file': nama_file,

    'header_size': i,

}
```

7. Pengecekan Kapasitas (*Payload*)

Sebelum melakukan *embedding*, program wajib memvalidasi apakah total byte muatan gabungan (header metadata + isi pesan) lebih kecil atau sama dengan kapasitas maksimal medium video. Proses penolakan otomatis (capacity rejection) akan langsung terjadi awal bila ukuran muatan berlebih untuk mencegah program berhenti tak terduga (*crash*).

Kapasitas maksimal bit sebuah video bergantung pada format skema yang dipilih:

1. Untuk Skema LSB Konvensional (Video AVI Lossless):

Rumus kapasitas dasarnya adalah hasil kali area kanvas, total dimensi panjang waktu (frame), dan Bit Per Pixel (BPP) yang dipilih:

$$\text{Kapasitas (bit)} = \text{Width} \times \text{Height} \times \text{Total_Frame} \times \text{BPP}_{\text{Skema}}$$

2. Untuk Skema Tangguh MP4 (Video MP4 Lossy):

Karena menggunakan metode Spatial Redundancy dengan blok 16×16 piksel demi ketahanan terhadap kompresi, algoritma akan memangkas fleksibilitas bit sebesar rasio blok. Selain itu, ada potongan tetap (potongan perlindungan ganda dan tanda identitas frame) sebesar 64-bit pada setiap framenya:

$$\text{Kapasitas (bit)} = ((\frac{\text{Width}}{16} \times \frac{\text{Height}}{16}) - 64) \times \text{Total_Frame}$$

Untuk mengonversi total kapasitas akhir pada kedua skema di atas ke dalam besaran Byte, hasil kalkulasi akhir hanya perlu dibagi dengan nilai 8 dalam pembagian bulat (integer division).

8. Pengukuran Kualitas (MSE, PSNR, dan Histogram)

Setelah proses *embedding* selesai, sistem melakukan evaluasi kualitas *stego-video* dibandingkan dengan video asli.

Pengukuran dilakukan menggunakan:

1. *Mean Squared Error* (MSE) untuk mengukur rata-rata perbedaan piksel.
2. *Peak Signal-to-Noise Ratio* (PSNR) untuk mengukur kualitas visual dalam satuan desibel.
3. Histogram RGB untuk membandingkan distribusi warna antara video asli dan *stego-video*.

Perhitungan dilakukan dengan membandingkan setiap *frame* pada video asli dan *stego-video*, kemudian diambil nilai rata-rata. Hasil perhitungan ditampilkan pada GUI sehingga pengguna dapat langsung mengevaluasi kualitas hasil penyisipan.

9. Steganografi *Robust* pada Video *Lossy* (MP4)

Format video MP4 menggunakan algoritma kompresi *lossy* (seperti H.264) yang membuang detail visual minor pada piksel untuk memperkecil ukuran file. Akibatnya, metode LSB (*Least Significant Bit*) tradisional akan hancur karena bit-bit terbawah dimodifikasi secara drastis saat proses kompresi (proses kuantisasi). Untuk menjawab permasalahan tersebut, dirancang sebuah skema khusus (mp4-robust) berbasis modifikasi domain spasial yang tahan banting (*lossy-robust*) dengan rancangan sebagai berikut:

1. Penyetelan Parameter Konstanta Spasial

Diterapkan dua konstanta utama: BLOCK_SIZE = 16 dan MASK_BIT = 64.

Satu bit payload diduplikasi menjadi 256 piksel pembentuk blok (16x16). Ini memastikan pesan memiliki cadangan / redundansi yang masif. Penempatan target modifikasi tidak berada di LSB pertama, tapi dipindahkan lebih tinggi ke bit ke-7 (representasi nilai 64). Bit ini terbukti cukup signifikan untuk lolos dari filter penghalusan kompresi (smoothing).

```
BLOCK_SIZE = 16 # 1 bit disebar ke 16x16 = 256 piksel biar
tahan kompresi
MASK_BIT = 64 # bit ke-7, cukup kuat tahan H.264 tapi
perubahannya ga kelihatan
```

2. Mekanisme Elevasi Bit dan Redundansi (Proses Embed)

Saat proses penyisipan berlangsung, seluruh piksel yang tercakup dalam radius blok 16x16 akan diubah secara kolektif sesuai nilai 1 atau 0. Modifikasi difokuskan murni pada channel spesifik (Indeks 0 atau channel Biru/Blue) untuk menyembunyikan distorsi secara visual dari tangkapan mata manusia.

```
for by in range(BLOCK_SIZE):
    for bx in range(BLOCK_SIZE):
        b = int(frame[y+by, x+bx, 0])
        if bit == 1:
            frame[y+by, x+bx, 0] = b | MASK_BIT
        else:
            frame[y+by, x+bx, 0] = b & ~MASK_BIT
```

3. Implementasi Kapsulasi dan Dummy Padding per-Frame

Setiap *frame* didesain dapat berdiri sendiri (*agnostik*) untuk mengantisipasi fenomena *frame-dropping* berkat penambahan *Frame Synchronization Marker* (Index dan Panjang bit). Selain itu, karena kompresi paling berpotensi merusak area sudut awal frame, digunakan

pelindung benturan berupa 32-bit dummy padding (bernilai nol) sebelum menyisipkan informasi aktual.

```
header_idx = [(frame_idx >> i) & 1 for i in range(16)]
header_len = [(len(chunk) >> i) & 1 for i in range(16)]
dummy = [0] * 32 # padding di depan biar distorsi awal
frame ga ngereject data

frame_data = dummy + header_idx + header_len + chunk
```

4. Esktraksi dengan Matematis Majority Voting

Untuk mengambil kembali pesan, program meniadakan kerentanan noise akibat *lossy compression* dengan rumus *probabilitas voting*. Dari total 256 piksel setiap blok matriks, program akan menotal (*vote*) jumlah piksel yang masih mempertahankan Mask Bit ke-7-nya. Jika lebih dari separuh batas kapasitas matriks (>128 piksel) bertahan, nilai blok dimutlakkan sebagai bit 1.

```
for frame in frames:
    bits = []

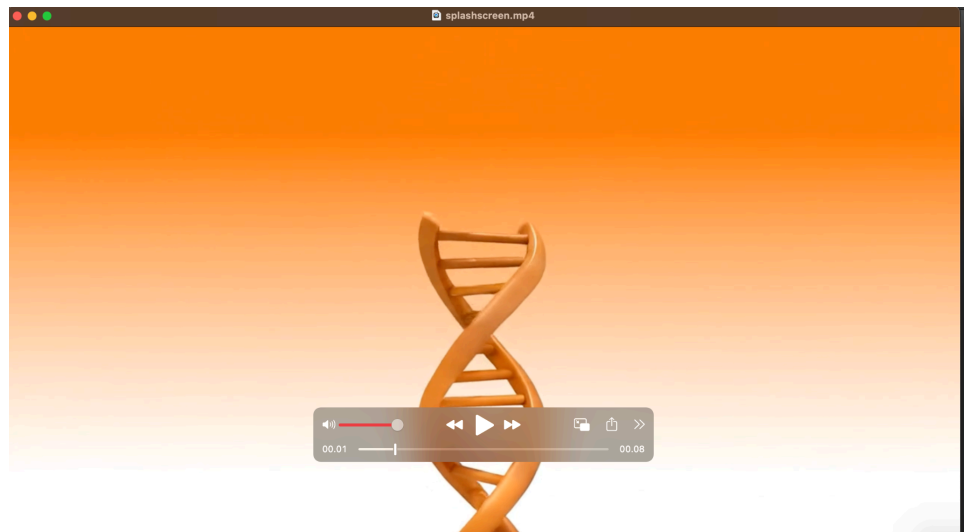
    for y in range(0, h - BLOCK_SIZE + 1, BLOCK_SIZE):
        for x in range(0, w - BLOCK_SIZE + 1, BLOCK_SIZE):
            vote_1 = 0
            for by in range(BLOCK_SIZE):
                for bx in range(BLOCK_SIZE):
                    b = int(frame[y+by, x+bx, 0])
                    if (b & MASK_BIT) != 0:
                        vote_1 += 1

            # majority vote: lebih dari setengah piksel di
            # blok ini bernilai 1?
            if vote_1 > (BLOCK_SIZE * BLOCK_SIZE) / 2:
                bits.append(1)
            else:
                bits.append(0)
```

5. Hasil Pengujian (Berhasil dan Gagal)

Pada tahap pengujian fitur penyisipan format MP4, kami menguji algoritma redundansi blok 16x16 ini pada beberapa video cover dengan karakteristik visual yang berbeda. Tujuannya adalah untuk melihat seberapa kuat perubahan piksel kita bertahan dari sistem kompresi bawaan MP4.

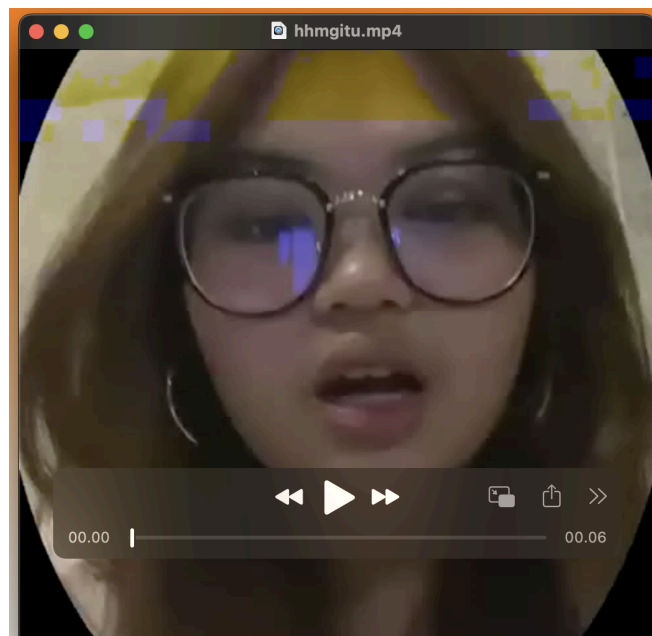
Pada pengujian pertama, kami menggunakan video animasi 3D (gambar DNA dengan latar oranye terang) yang berukuran sekitar 567 KB.



Gambar 2.1 Video Cover Pengujian MP4 1.

Hasilnya, pesan teks berhasil disisipkan dan diekstrak kembali dengan mulus tanpa ada pesan error.

Namun, pada pengujian kedua, kami menggunakan video rekaman kamera biasa (menampilkan wajah dengan pencahayaan yang lebih gelap dan detail warna yang lebih padat).



Gambar 2.2 Video Cover Pengujian MP4 2.

Pada video kedua ini, ekstraksi gagal total dan program mengembalikan error berupa angka acak saat membaca rentetan biner

header awal (misalnya error "Jenis pesan tidak valid di header: 28" atau "Mode code: 48").

Menariknya, kalau kita perhatikan hasil akhir video kedua, bagian atas video malah memunculkan distorsi kotak-kotak keunguan (seperti glitch) yang sangat jelas terlihat oleh mata. Distorsi ini adalah wujud fisik dari pesan rahasia kita yang disisipkan di bit ke-7 pada channel warna biru.

6. Analisis Limitasi

Dari perbedaan hasil di atas, kami menemukan limitasi yang sangat jelas dari metode steganografi spasial untuk format MP4. Keberhasilan penyisipan ini ternyata sangat bergantung pada profil warna video penutupnya, bukan cuma sekadar ukuran file.

Kegagalan pada video kedua terjadi karena cara kerja kompresi dasar MP4 (algoritma H.264). Sistem kompresi ini punya kebiasaan membuang atau menghaluskan (*blurring*) detail warna sekitarnya untuk menghemat ukuran file, apalagi kalau area tersebut warnanya gelap, kusam, atau terlalu banyak teksturnya.

Karena encoder memadukan dan memudarkan piksel-piksel biru buatan kita, blok 16x16 yang memegang data ikut rusak. Saat program mencoba melakukan ekstraksi dan menghitung batas *majority vote* (syarat minimal 128 piksel benar), jumlahnya kurang dari target. Akibatnya, ada 1-2 nilai bit yang meleset. Hanya karena satu bit saja yang berubah atau bergeser di awal pembacaan metadata header, sisa panjang 32-bit selanjutnya jadi berantakan, makanya program membaca nilai yang tidak masuk akal (seperti angka 3 atau 48).

7. Kesimpulan

Batasan utama dari rancangan ini adalah ketahanannya kalah kalau cover video aslinya didominasi warna gelap atau punya detail rumit yang memicu algoritma MP4 merusak warnanya. Algoritma ini baru bisa bekerja maksimal kalau kita memilih video dengan pencahayaan terang dan solid (seperti contoh video DNA oranye) karena kompresi MP4 tidak akan terlalu banyak mengubah dan merusak susunan warna aslinya.

III. Pengujian Program dan Analisis Hasil

1. Semua Kombinasi Konfigurasi

Pengujian dilakukan menggunakan antarmuka GUI pada tab *Embed* dengan memasukkan video *cover* yang sama dan *payload* berupa teks. Parameter yang divariasikan meliputi skema LSB, mode penyisipan, serta penggunaan enkripsi A5/1.

Payload yang digunakan pada pengujian ini adalah:



Gambar 3.1 *File Payload* sendi.png

a) Konfigurasi Pengujian

Total pengujian yang dilakukan adalah sebanyak **12 kombinasi**. Setiap pengujian menggunakan video *cover* dan *payload* yang sama agar hasil yang diperoleh dapat dibandingkan secara konsisten. Untuk mode random digunakan *stego-key*, sedangkan untuk enkripsi digunakan *key A5/1*.

Konfigurasi lengkap dari setiap pengujian ditunjukkan pada tabel berikut:

Tabel 3.1 Tabel Konfigurasi Pengujian dengan Berbagai Kombinasi

Kode Pengujian	Skema LSB	Metode Penyisipan	Enkripsi	<i>Stego-Key</i>	<i>Key A5/1</i>
T1	1-1-1	<i>Sequential</i>	<i>Off</i>	-	-
T2	1-1-1	<i>Sequential</i>	<i>On</i>	-	123456
T3	1-1-1	<i>Random</i>	<i>Off</i>	key123	-
T4	1-1-1	<i>Random</i>	<i>On</i>	key123	123456
T5	3-3-2	<i>Sequential</i>	<i>Off</i>	-	-
T6	3-3-2	<i>Sequential</i>	<i>On</i>	-	123456

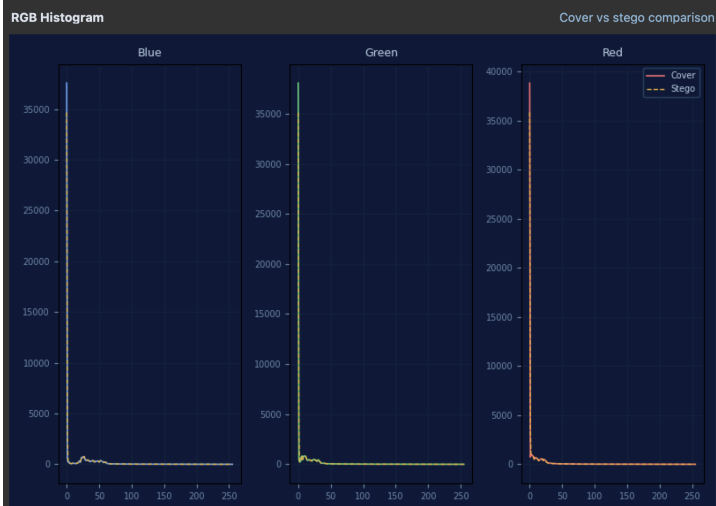
T7	3-3-2	Random	Off	key123	-
T8	3-3-2	Random	On	key123	123456
T9	4-4-4	Sequential	Off	-	-
T10	4-4-4	Sequential	On	-	123456
T11	4-4-4	Random	Off	key123	-
T12	4-4-4	Random	On	key123	123456

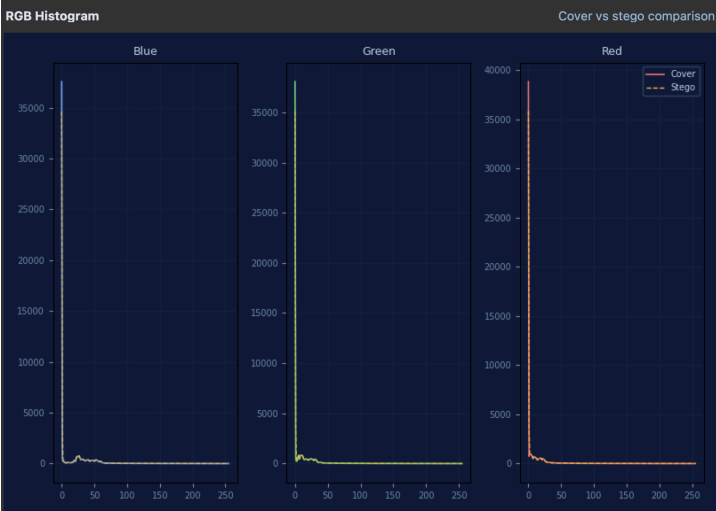
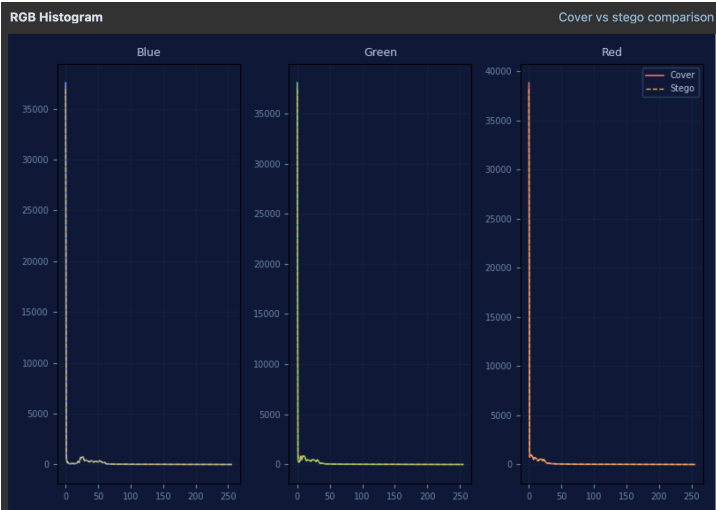
b) Hasil Pengujian

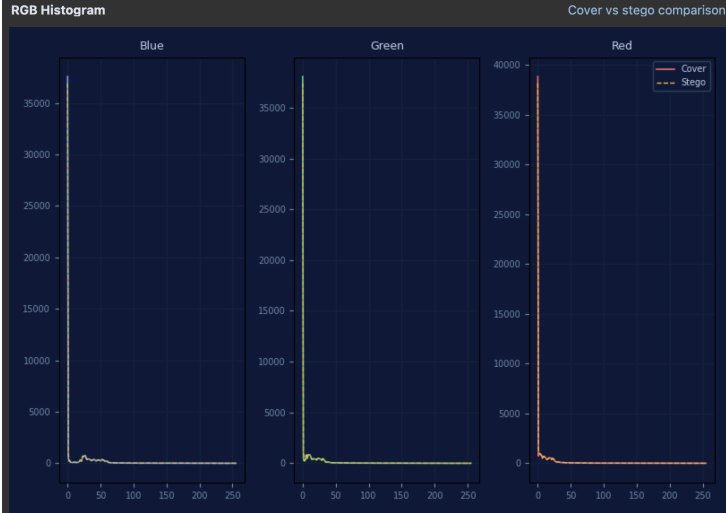
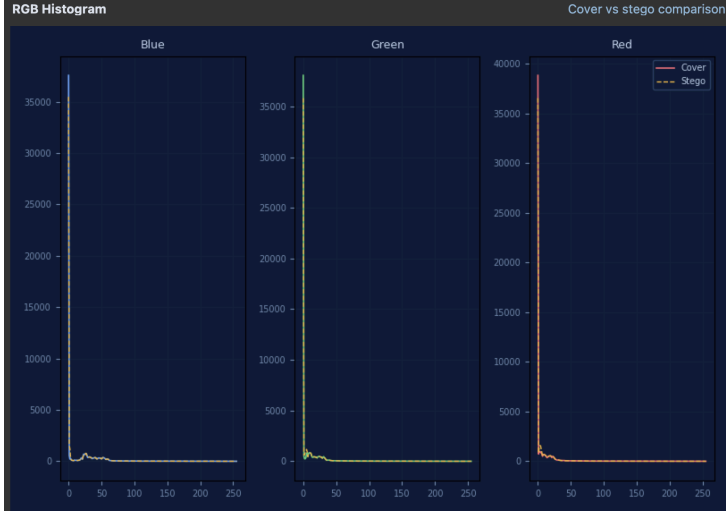
Hasil pengujian diukur menggunakan dua metrik utama, yaitu *Mean Squared Error* (MSE) dan *Peak Signal-to-Noise Ratio* (PSNR). Selain itu, dilakukan juga perbandingan histogram untuk melihat perubahan distribusi warna.

Ringkasan hasil pengujian ditunjukkan pada tabel berikut:

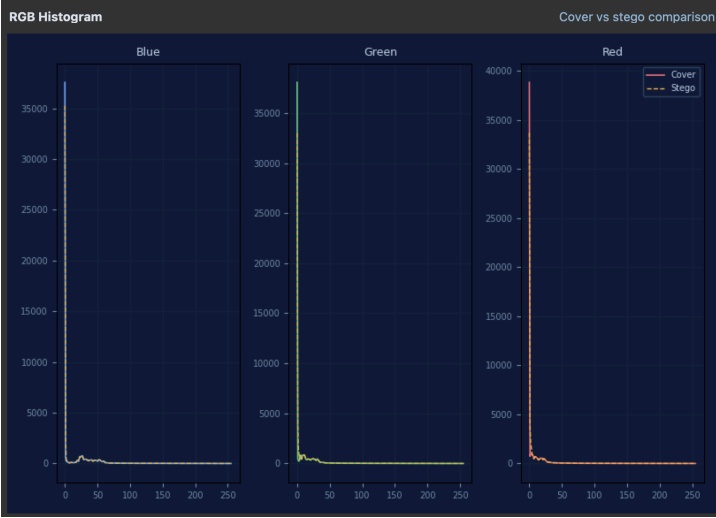
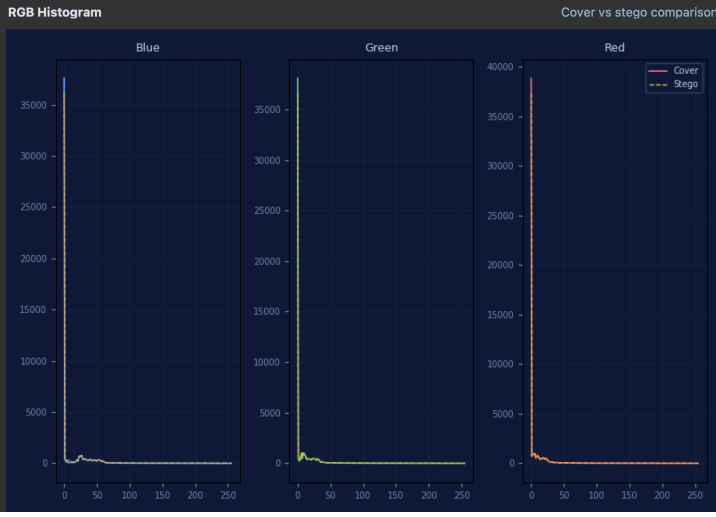
Tabel 3.2 Tabel Hasil Pengujian dengan Berbagai Kombinasi

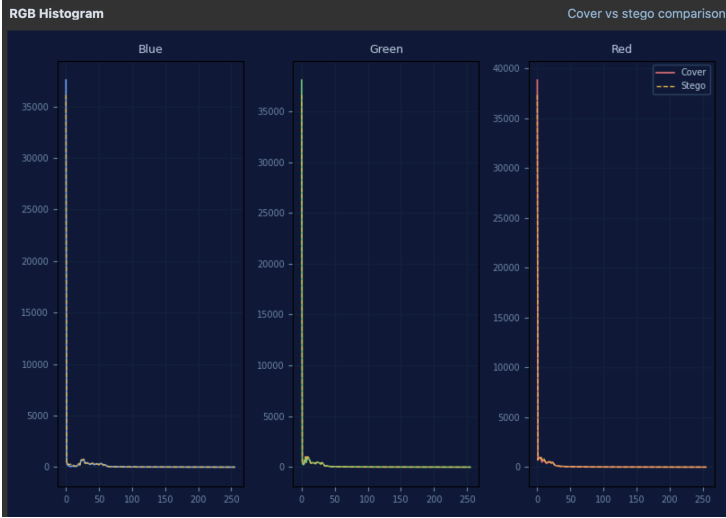
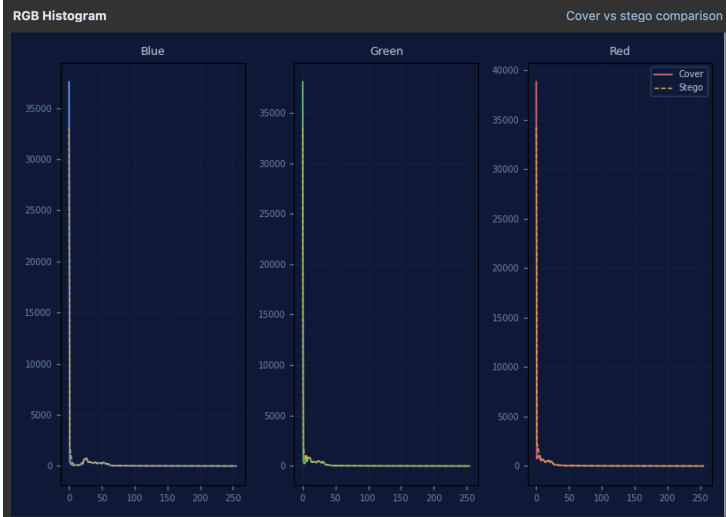
Kode Pengujian	MSE	PSNR (dB)
T1	0.07781	59.22
 Gambar 3.2 Histogram Pengujian T1		
T2	0.07789	59.22

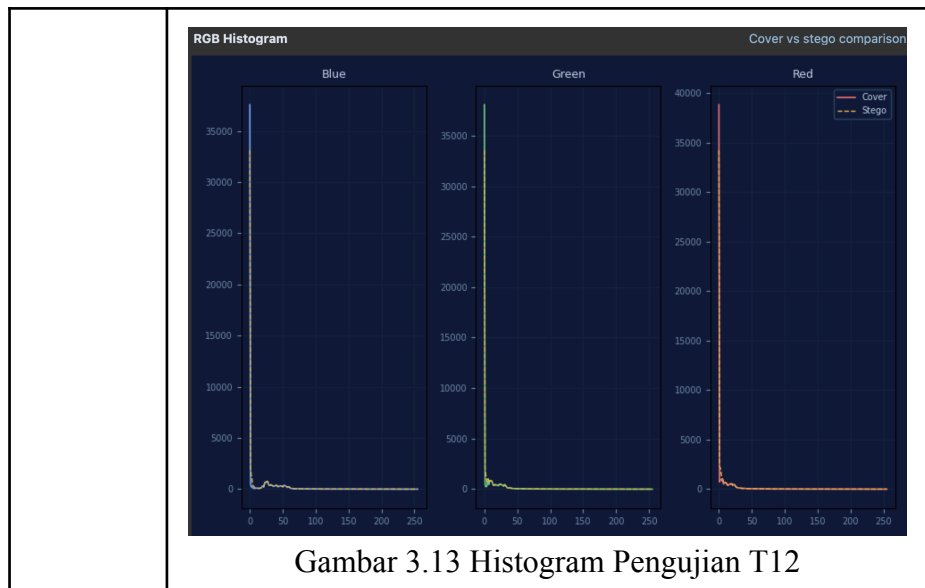
	 Gambar 3.3 Histogram Pengujian T2	
T3	0.07786	59.22
	 Gambar 3.4 Histogram Pengujian T3	
T4	0.07792	59.21

	 Gambar 3.5 Histogram Pengujian T4	
T5	0.65399	49.98
	 Gambar 3.6 Histogram Pengujian T5	
T6	0.65165	49.99

	RGB Histogram Cover vs stego comparison Blue Green Red Gambar 3.7 Histogram Pengujian T6	
T7	0.65236	49.99
	RGB Histogram Cover vs stego comparison Blue Green Red Gambar 3.8 Histogram Pengujian T7	
T8	0.65109	49.99

	 Gambar 3.9 Histogram Pengujian T8	
T9	2.53698	44.09
	 Gambar 3.10 Histogram Pengujian T9	
T10	2.52336	44.11

	 Gambar 3.11 Histogram Pengujian T10	
T11	2.54542	44.07
	 Gambar 3.12 Histogram Pengujian T11	
T12	2.53675	44.09



Gambar 3.13 Histogram Pengujian T12

Secara umum diperoleh hasil sebagai berikut:

- Skema 1-1-1 menghasilkan MSE paling kecil (sekitar 0.077) dan PSNR tertinggi (sekitar 59 dB)
- Skema 3-3-2 menghasilkan MSE sedang (sekitar 0.65) dan PSNR sekitar 50 dB
- Skema 4-4-4 menghasilkan MSE terbesar (sekitar 2.53) dan PSNR terendah (sekitar 44 dB)

Hasil ini menunjukkan adanya variasi kualitas yang cukup signifikan antar skema yang digunakan.

c) Analisis Hasil

Berdasarkan hasil pengujian, dapat diamati bahwa **skema LSB merupakan faktor utama yang mempengaruhi kualitas stego-video**.

Skema **1-1-1** menghasilkan nilai **MSE paling kecil** (sekitar 0.077) dan PSNR tertinggi (sekitar 59 dB), yang menunjukkan bahwa **distorsi yang dihasilkan sangat kecil**. Hal ini disebabkan karena jumlah bit yang dimodifikasi pada setiap piksel relatif sedikit, sehingga perubahan visual hampir tidak terlihat.

Sebaliknya, skema **4-4-4** menghasilkan nilai **MSE terbesar** (sekitar 2.53) dan **PSNR terendah** (sekitar 44 dB). Hal ini menunjukkan bahwa semakin banyak bit yang dimodifikasi, semakin besar distorsi yang terjadi pada video.

Skema **3-3-2** berada **di antara kedua skema tersebut** dengan nilai MSE sekitar 0.65 dan PSNR sekitar 50 dB, sehingga dapat dianggap sebagai kompromi antara kapasitas penyisipan dan kualitas visual. Dengan demikian, terlihat adanya **trade-off antara kapasitas**

dan kualitas, di mana peningkatan jumlah bit yang disisipkan akan menyebabkan penurunan kualitas visual.

Dari sisi metode penyisipan, **tidak ditemukan perbedaan signifikan** antara mode *sequential* dan *random* terhadap nilai MSE maupun PSNR. Hal ini disebabkan karena kedua metode hanya berbeda pada urutan pemilihan piksel, bukan jumlah bit yang dimodifikasi.

Penggunaan enkripsi A5/1 juga tidak memberikan pengaruh terhadap kualitas visual video. Hal ini karena enkripsi hanya mengubah isi *payload*, tanpa memengaruhi jumlah bit yang disisipkan ke dalam video.

Berdasarkan hasil **histogram** yang ditampilkan, distribusi warna antara video asli dan stego **tidak menunjukkan perbedaan yang signifikan**. Hal ini menunjukkan bahwa proses penyisipan tetap mempertahankan karakteristik visual video dan memenuhi aspek *imperceptibility*.

d) Pengujian Integritas Berkas

Pengujian integritas dilakukan untuk memastikan bahwa *payload* yang disisipkan ke dalam video dapat diekstraksi kembali tanpa mengalami perubahan data. Pengujian ini dilakukan dengan membandingkan nilai *hash* MD5 antara *file* asli dan *file* hasil ekstraksi.

```
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/send1.png
MD5 (/Users/alicia/Downloads/send1.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T1.png
MD5 (/Users/alicia/Downloads/T1.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T2.png
MD5 (/Users/alicia/Downloads/T2.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T3.png
MD5 (/Users/alicia/Downloads/T3.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T4.png
MD5 (/Users/alicia/Downloads/T4.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T5.png
MD5 (/Users/alicia/Downloads/T5.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T6.png
MD5 (/Users/alicia/Downloads/T6.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T7.png
MD5 (/Users/alicia/Downloads/T7.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T8.png
MD5 (/Users/alicia/Downloads/T8.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T9.png
MD5 (/Users/alicia/Downloads/T9.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T10.png
MD5 (/Users/alicia/Downloads/T10.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T11.png
MD5 (/Users/alicia/Downloads/T11.png) = f4186e6e7ed2a147f37c6cfae778c3c
(base) alicia@alicias-macbook-3 ~ % md5 /Users/alicia/Downloads/T12.png
MD5 (/Users/alicia/Downloads/T12.png) = f4186e6e7ed2a147f37c6cfae778c3c
```

Gambar 3.14 Hasil Pengujian Integritas Berkas dengan Berbagai Kombinasi

Tabel 3.3 Tabel Hasil Pengujian Integritas Berkas dengan Berbagai Kombinasi

Kode Pengujian	MD5 Asli (sendi.png)	MD5 Ekstraksi (TX.png)	Status
T1	f4186e6e7ed2a147f37c6cffae778c3c	f4186e6e7ed2a147f37c6cffae778c3c	Identik
T2		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T3		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T4		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T5		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T6		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T7		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T8		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T9		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T10		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T11		f4186e6e7ed2a147f37c6cffae778c3c	Identik
T12		f4186e6e7ed2a147f37c6cffae778c3c	Identik

Berdasarkan hasil pada Tabel 3.3, diperoleh bahwa nilai *hash* MD5 antara *file* asli (sendi.png) dan seluruh file hasil ekstraksi adalah **identik**, yaitu **f4186e6e7ed2a147f37c6cffae778c3c**. Kesamaan nilai *hash* ini menunjukkan bahwa **tidak terjadi perubahan data** pada *payload* selama proses *embedding* dan *extraction*. Dengan kata lain, sistem mampu mengembalikan *file* hasil ekstraksi secara utuh hingga pada level *byte*.

Hasil ini juga menunjukkan bahwa seluruh parameter yang digunakan, termasuk skema LSB, metode penyisipan, *stego-key* pada mode *random*, serta *key* A5/1 pada mode enkripsi, telah digunakan

secara konsisten antara proses *embedding* dan *extraction*. Selain itu, hasil ini membuktikan bahwa penggunaan enkripsi A5/1 tidak mempengaruhi integritas data selama proses dekripsi dilakukan dengan parameter yang benar. Dengan demikian, sistem **berhasil memenuhi aspek *recovery***, yaitu kemampuan untuk mengembalikan pesan secara utuh tanpa kehilangan maupun perubahan informasi.

2. Penyisipan yang Melebihi Batas Kapasitas Sisip

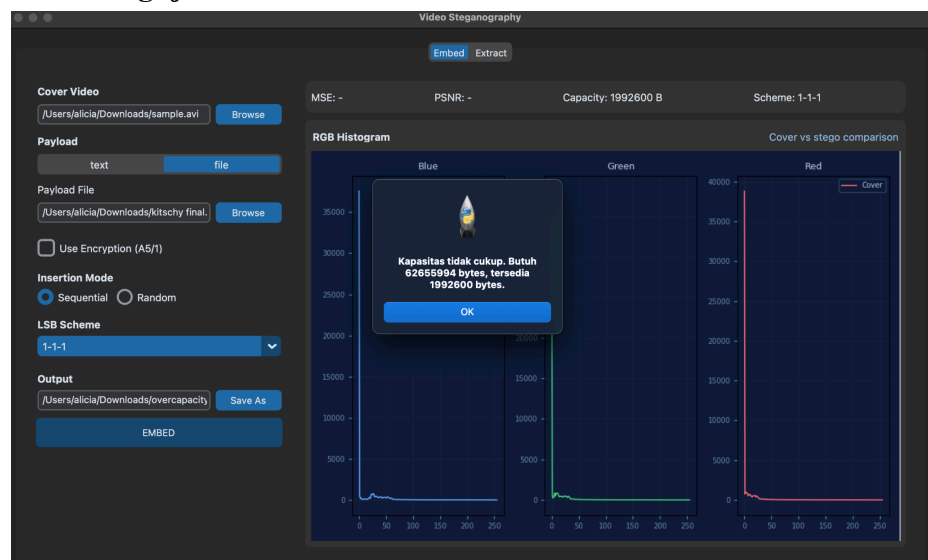
a) Konfigurasi Pengujian

Berikut merupakan konfigurasi lengkap dari penyisipan yang melebihi batas kapasitas sisip:

Tabel 3.4 Tabel Konfigurasi Pengujian dengan Berkas Melebihi Batas Kapasitas Sisip

Kode Pengujian	Skema LSB	Metode Penyisipan	Enkripsi	Kapasitas <i>Cover Video</i>	Kapasitas Berkas
T14	1-1-1	<i>Sequential</i>	<i>Off</i>	1992600 bytes	62655994 bytes

b) Hasil Pengujian



Gambar 3.15 Hasil Pengujian dengan Berkas Melebihi Batas Kapasitas Sisip

Hasil pengujian ditunjukkan pada Gambar 3.15, di mana sistem menampilkan pesan bahwa kapasitas video tidak mencukupi untuk menyisipkan *payload* yang diberikan.

Berdasarkan hasil tersebut, diketahui bahwa:

- Kapasitas maksimum video adalah sebesar **1.992.600 bytes**

- Ukuran *payload* yang akan disisipkan adalah sebesar **62.655.994 bytes**

Karena ukuran *payload* jauh melebihi kapasitas yang tersedia, sistem secara otomatis menolak proses embedding dan menampilkan pesan kesalahan berupa “*Kapasitas tidak cukup*”. Proses penyisipan tidak dilanjutkan sehingga tidak terjadi perubahan pada video *cover*.

c) Analisis Hasil

Hasil pengujian menunjukkan bahwa sistem telah memiliki mekanisme validasi kapasitas yang berjalan dengan baik. Sistem mampu mendeteksi kondisi ketika ukuran *payload* melebihi kapasitas maksimum sebelum proses embedding dilakukan.

Perbandingan antara kapasitas video dan ukuran *payload* yang sangat jauh (sekitar lebih dari 30 kali lipat) menunjukkan bahwa sistem melakukan **pengecekan secara akurat dan tidak memaksakan proses penyisipan**. Hal ini penting untuk mencegah terjadinya kesalahan seperti data terpotong atau kegagalan proses *embedding*.

Selain itu, validasi ini juga memastikan bahwa kualitas video tetap terjaga, karena tidak ada proses modifikasi yang dilakukan ketika kapasitas tidak mencukupi.

Dengan demikian, dapat disimpulkan bahwa sistem telah **memenuhi aspek *capacity***, yaitu kemampuan untuk menghitung dan membatasi jumlah data yang dapat disisipkan secara tepat serta menangani kondisi *overflow* dengan baik.

3. Penyisipan dengan Berbagai Jenis Berkas

a) Konfigurasi Pengujian

Total pengujian yang dilakukan adalah sebanyak **8 kombinasi**, dengan tiap pengujian menggunakan video *cover* yang sama agar hasil yang diperoleh dapat dibandingkan secara konsisten. Untuk *payload* digunakan berkas teks dengan jenis yang berbeda. Konfigurasi lengkap dari setiap pengujian ditunjukkan pada tabel berikut.

Tabel 3.5 Tabel Konfigurasi Pengujian dengan Berbagai Jenis Berkas

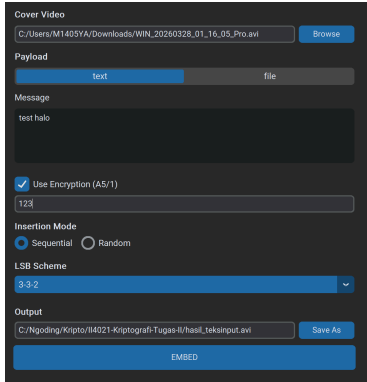
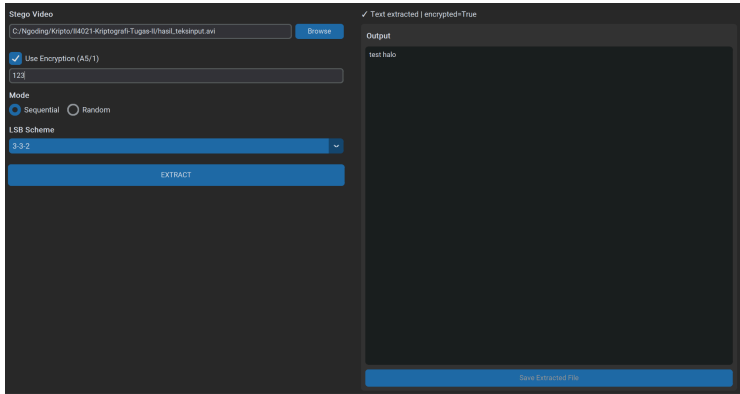
Kode Pengujian	Skema LSB	Metode Penyisipan	Enkripsi	Jenis Berkas Teks
T15	3-3-2	<i>Sequential</i>	<i>On</i>	Teks berdasarkan <i>input</i> pengguna (<i>string</i>)
T16	3-3-2	<i>Sequential</i>	<i>On</i>	.txt

T17	3-3-2	<i>Sequential</i>	<i>On</i>	.docx
T18	3-3-2	<i>Sequential</i>	<i>On</i>	.pdf
T19	3-3-2	<i>Sequential</i>	<i>On</i>	.xlsx
T20	3-3-2	<i>Sequential</i>	<i>On</i>	.png
T21	3-3-2	<i>Sequential</i>	<i>On</i>	.jpg
T22	3-3-2	<i>Sequential</i>	<i>On</i>	.exe

b) Hasil Pengujian

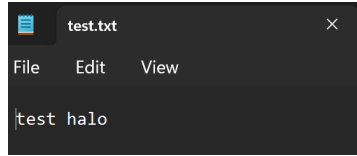
Hasil pengujian dilihat dari keberhasilan dari proses *embedding* dan *extracting*, dengan mengecek kesamaan *file* yang digunakan dalam *embedding* dan hasil *extracting*. Ringkasan hasil pengujian ditunjukkan pada tabel berikut.

Tabel 3.6 Tabel Konfigurasi Pengujian dengan Berbagai Jenis Berkas

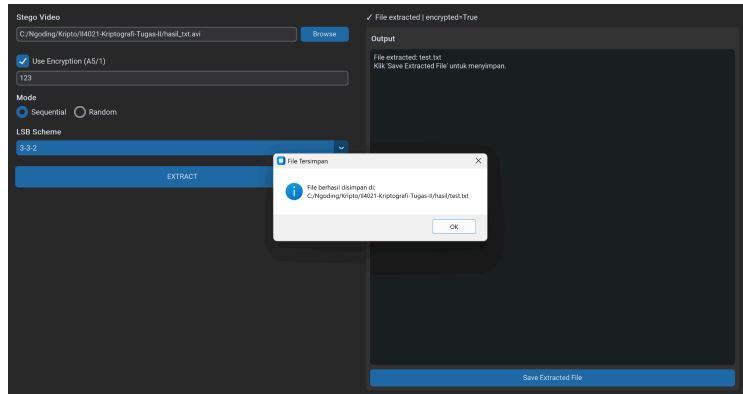
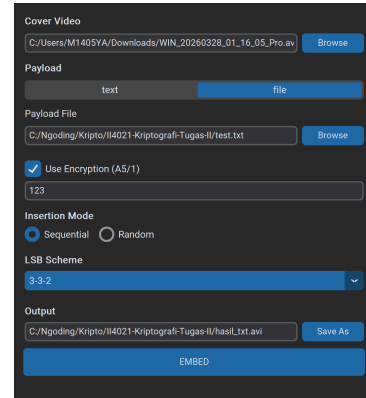
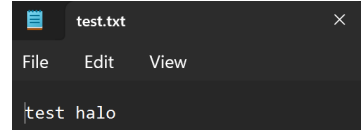
Kode Pengujian	File yang Digunakan untuk <i>Embedding</i>	File Hasil <i>Extracting</i>
T15	Teks yang diinput dalam <i>textbox</i> : “test halo”	Berupa teks: “test halo”
 		

T16

File test.txt yang berisi:

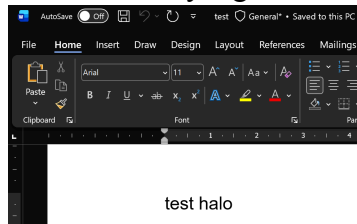


File dengan nama sama (test.txt) yang berisi:

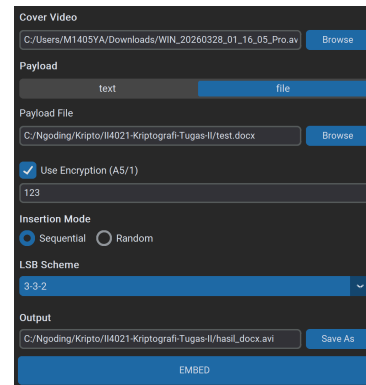
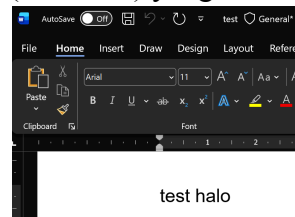


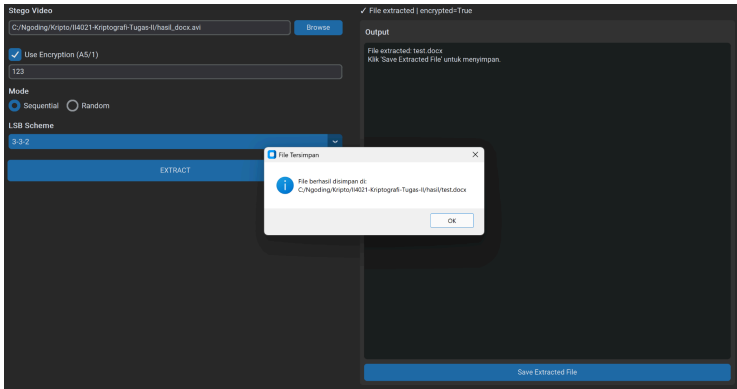
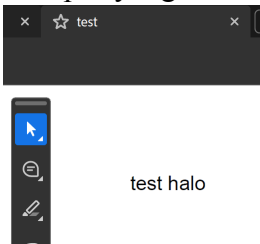
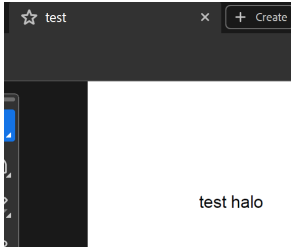
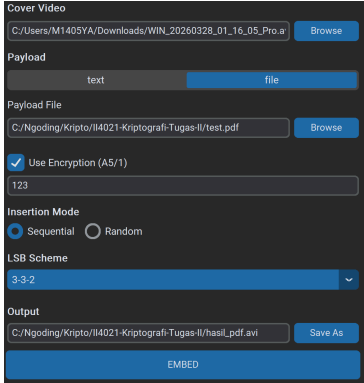
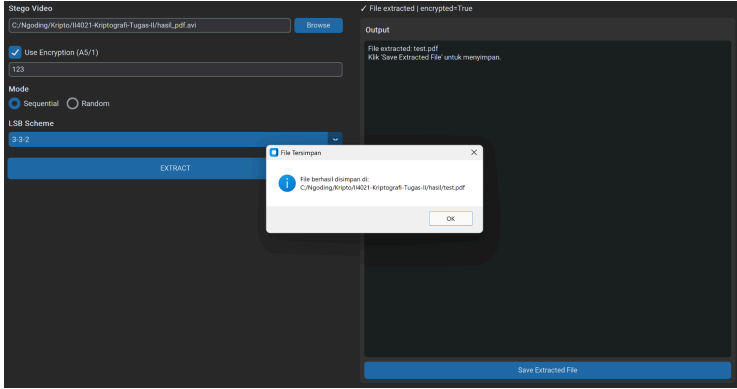
T17

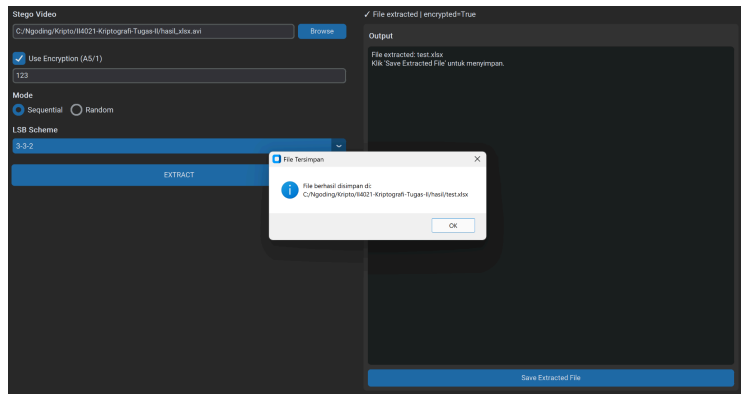
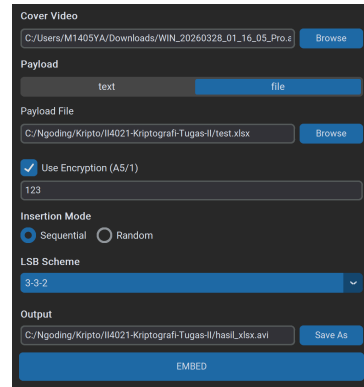
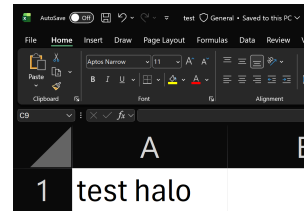
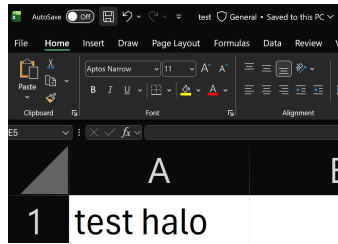
File test.docx yang berisi:



File dengan nama sama (test.docx) yang berisi:

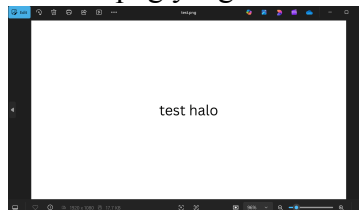


		
T18	<i>File test.pdf yang berisi:</i> 	<i>File dengan nama sama (test.pdf) yang berisi:</i> 
	 	
T19	<i>File test.xlsx yang berisi:</i>	<i>File dengan nama sama (test.xlsx) yang berisi:</i>

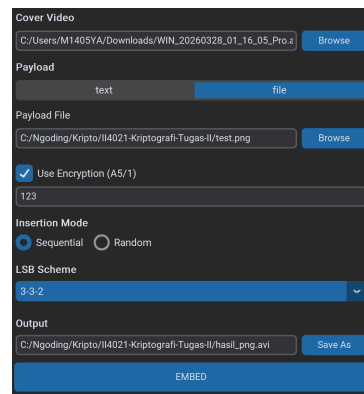
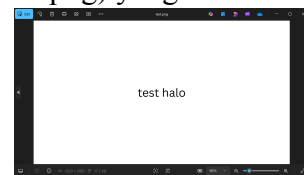


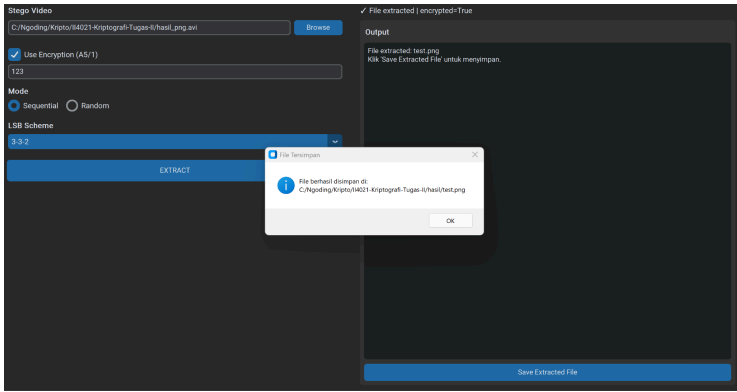
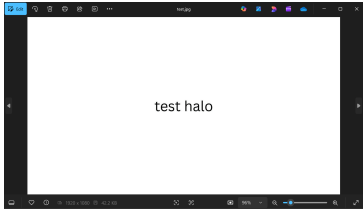
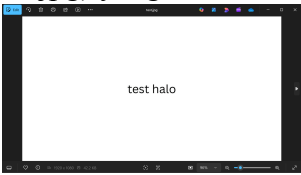
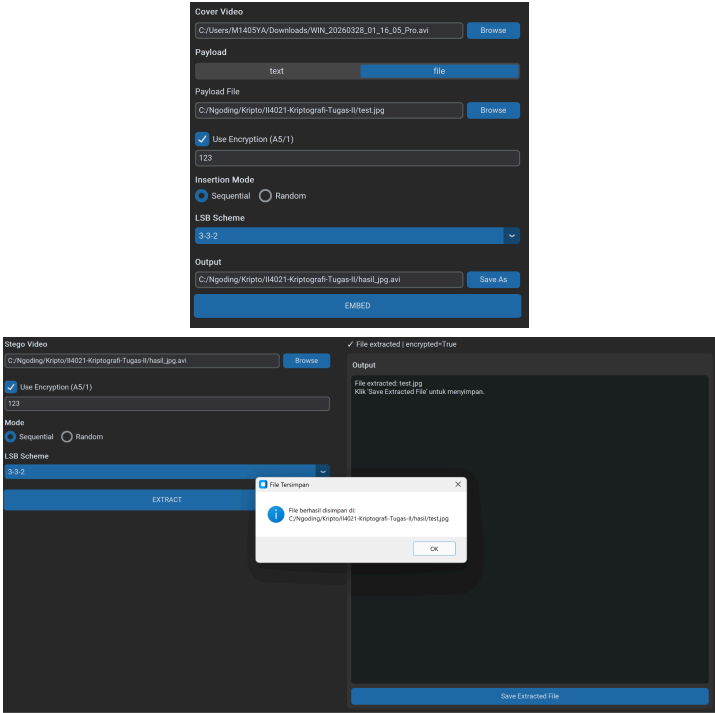
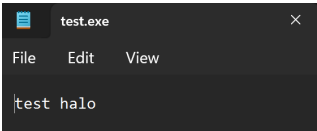
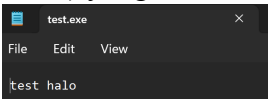
T20

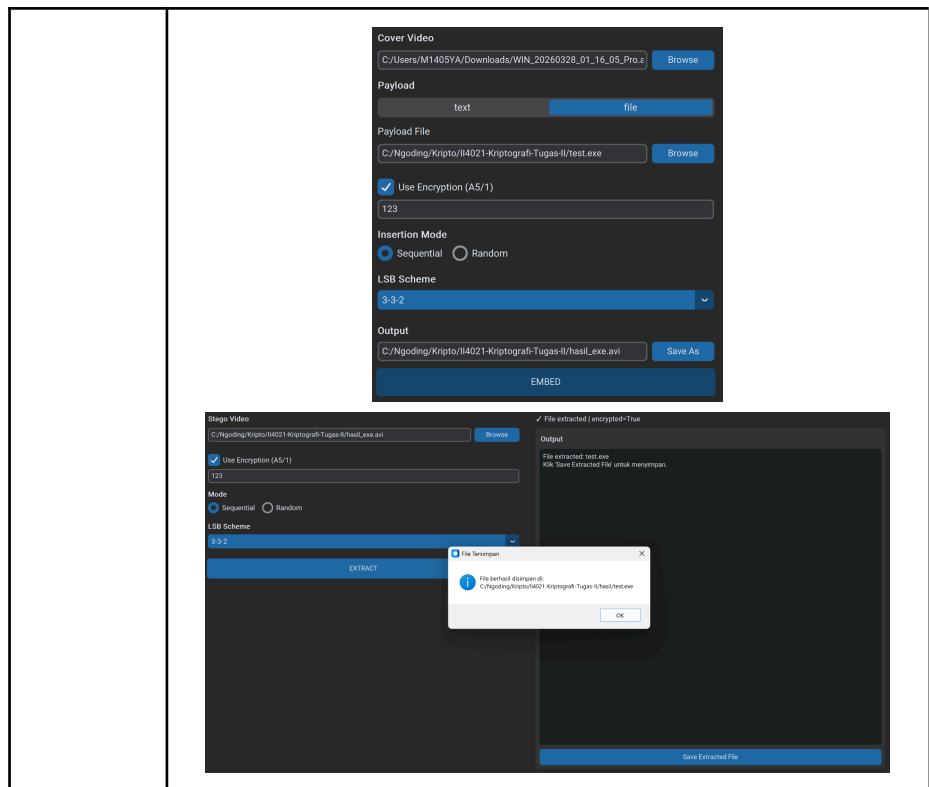
File test.png yang berisi:



File dengan nama sama (test.png) yang berisi:



		
T21	<i>File test.jpg yang berisi:</i> 	<i>File dengan nama sama (test.jpg) yang berisi:</i> 
		
T22	<i>File test.exe yang berisi:</i> 	<i>File dengan nama sama (test.exe) yang berisi:</i> 



Secara umum diperoleh hasil bahwa seluruh pengetesan pada berbagai berkas *file* berhasil.

c) Analisis Hasil

Hasil pengujian menunjukkan bahwa sistem telah memiliki mekanisme yang dapat memproses pesan dalam bentuk apapun, baik *file* maupun teks biasa. *File* yang bisa di-*handle* pun dalam bentuk apapun, menunjukkan bahwa sistem mampu **melakukan proses terhadap pesan yang ingin disisipkan, dalam format apapun**. Hal ini penting untuk fleksibilitas dalam *input* pengguna. Selain itu, hasil juga menunjukkan bahwa *file* hasil ekstraksi dari *video* yang telah di-*embed* oleh *file* tertentu, hasilnya identik, baik dari nama *file* hingga konten isinya. Hal ini menunjukkan bahwa sistem mampu **melakukan pemulihan file hasil embedding, memenuhi aspek recovery**.

d) Pengujian Integritas Berkas

Pengujian integritas dilakukan untuk memastikan bahwa *payload* yang disisipkan ke dalam *video* dapat diekstraksi kembali tanpa mengalami perubahan data. Pengujian ini dilakukan dengan membandingkan nilai *hash* MD5 antara *file* asli dan *file* hasil ekstraksi.

```

PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.txt md5
MD5 hash of test.txt:
b93ebc8cce33dd57a1edd8dccb2f2b0c
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.pdf md5
MD5 hash of test.pdf:
eab2818efc18dc0e59fb998f18238b51
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.docx md5
MD5 hash of test.docx:
67b241469833c989ea7fbcf1bc074088
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.xlsx md5
MD5 hash of test.xlsx:
73164d10fd67e3caff08c51967a055af
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.jpg md5
MD5 hash of test.jpg:
1dfd1b65759436ccc9c323bf06c309fd
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.png md5
MD5 hash of test.png:
b150c116ce0ebee7013586f7ca56ea04
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> certutil -hashfile test.exe md5
MD5 hash of test.exe:
b93ebc8cce33dd57a1edd8dccb2f2b0c
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.txt:
b93ebc8cce33dd57a1edd8dccb2f2b0c
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.docx:
67b241469833c989ea7fbcf1bc074088
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.pdf:
eab2818efc18dc0e59fb998f18238b51
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.xlsx:
73164d10fd67e3caff08c51967a055af
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.jpg:
1dfd1b65759436ccc9c323bf06c309fd
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.png:
b150c116ce0ebee7013586f7ca56ea04
CertUtil: -hashfile command completed successfully.
PS C:\Ngoding\Kripto\II4021-Kriptografi-Tugas-II> cd .\hasil\
MD5 hash of test.exe:
b93ebc8cce33dd57a1edd8dccb2f2b0c
CertUtil: -hashfile command completed successfully.

```

Gambar 3.14 Hasil Pengujian Integritas Berkas dengan Berbagai Jenis Berkas

Tabel 3.7 Tabel Hasil Pengujian Integritas Berkas dengan Berbagai Jenis Berkas

Kode Pengujian	MD5 Asli	MD5 Ekstraksi	Status
T16 (test.txt)	b93ebc8cce33dd57a1edd8dccb2f2b0c	b93ebc8cce33dd57a1edd8dccb2f2b0c	Identik
T17 (test.docx)	67b241469833c989ea7fbcf1bc074088	67b241469833c989ea7fbcf1bc074088	Identik
T18 (test.pdf)	eab2818efc18dc0e59fb998f18238b51	eab2818efc18dc0e59fb998f18238b51	Identik
T19 (test.xlsx)	73164d10fd67e3caff08c51967a055af	73164d10fd67e3caff08c51967a055af	Identik
T20 (test.jpg)	1dfd1b65759436ccc9c323bf06c309fd	1dfd1b65759436ccc9c323bf06c309fd	Identik

T21 (test.png)	b150c116ce0ebee701 3586f7ca56ea04	b150c116ce0ebee701 3586f7ca56ea04	Identik
T22 (test.exe)	b93ebc8cce33dd57a1 edd8dccf2f2b0c	b93ebc8cce33dd57a1 edd8dccf2f2b0c	Identik

Berdasarkan hasil pada Tabel 3.7, diperoleh bahwa nilai *hash* MD5 antara *file* asli dan seluruh *file* hasil ekstraksi adalah **identik**. Kesamaan nilai *hash* ini menunjukkan bahwa **tidak terjadi perubahan data** pada *payload* selama proses *embedding* dan *extraction*. Dengan kata lain, sistem mampu mengembalikan *file* hasil ekstraksi secara utuh hingga pada level *byte*. Hasil ini juga membuktikan bahwa penggunaan enkripsi A5/1 tidak mempengaruhi integritas data selama proses dekripsi dilakukan dengan parameter yang benar. Dengan demikian, sistem **berhasil memenuhi aspek recovery**, yaitu kemampuan untuk mengembalikan pesan secara utuh tanpa kehilangan maupun perubahan informasi.

IV. Kesimpulan Hasil Implementasi

1. Kesimpulan

Berdasarkan hasil pengembangan sistem steganografi LSB pada berkas *video* AVI dengan enkripsi A5/1 ini, terdapat beberapa *highlights*:

- Sistem berhasil menggabungkan algoritma *stream cipher* A5/1 dengan metode steganografi LSB.** Steganografi menyembunyikan pesan, sedangkan enkripsi memastikan pesan tetap rahasia meski terekstraksi. Kombinasi kedua aspek ini menciptakan keamanan tinggi.
- Seluruh data dapat diekstraksi secara utuh dan identik dengan *file* asli**, dengan bukti kesamaan nilai *hash* MD5 dari hasil *file* sebelum dan setelah steganografi. Integritas data dijaga dengan format *video* dan *header* metadata yang dinamis.
- Terdapat hubungan *trade-off* antara kapasitas penyisipan dengan kualitas visual *stego-video*.** Skema 1-1-1 menghasilkan kualitas visual terbaik dengan PSNR tertinggi (~59 dB), tetapi kapasitasnya rendah. Sedangkan skema 4-4-4 memberikan kapasitas yang sangat besar, tetapi distorsi visualnya cukup terlihat, dengan PSNR terendah, yaitu ~44 dB. Skema 3-3-2 pun menjadi skema dengan kapasitas dan kualitas visual yang paling optimal.
- Penggunaan metode penyisipan *random* dengan enkripsi memiliki kelebihan keamanan yang lebih tinggi, dengan harga yang murah.** Penyisipan *random* meningkatkan keamanan distribusi pesan tanpa memengaruhi kualitas visual secara jelas. Penggunaan enkripsi juga tidak menambah beban distorsi pada *video* karena hanya mengubah nilai bit tanpa menambah jumlahnya.

2. Saran

Untuk pengembangan lebih lanjut, terdapat beberapa hal yang bisa dipertimbangkan:

- a) **Menggunakan algoritma *stream cipher* yang lebih kuat** dengan keamanan yang lebih tinggi, seperti AES-CTR yang bisa meningkatkan level keamanan data.
- b) **Mengoptimasi kecepatan proses**, dengan menggunakan teknik *multithreading* yang bisa membantu mempercepat pengacakan piksel dalam skema *random* dengan enkripsi, utamanya untuk video dengan resolusi tinggi dan durasi yang panjang.

V. Daftar Pustaka

- [1] Petitcolas, F.A.P., Anderson, R.J., & Kuhn, M.G. (1999). Information Hiding — A Survey. *Proceedings of the IEEE*, 87(7), 1062–1078.
- [2] Johnson, N.F., & Jajodia, S. (1998). Exploring Steganography: Seeing the Unseen. *IEEE Computer*, 31(2), 26–34.
- [3] Microsoft Corporation. (1992). AVI RIFF File Reference. Microsoft Developer Network Documentation.
- [4] Currie, D.L. & Irvine, C.E. (1996). Surmounting the effects of lossy compression on Steganography. 19th National Information Systems Security Conference.
- [5] Nados, W.L., & Rasheed, N.A. (2019). Hiding Text in AVI Video File by Method of Least Significant Bit.

- [6] R. Awati, "what is a stream cipher?," *SearchSecurity*, Dec. 10, 2024. <https://www.techtarget.com/searchsecurity/definition/stream-cipher>
- [7] "Stream cipher," *Training Camp*, 2024. <https://trainingcamp.com/glossary/stream-cipher/> (accessed Mar. 28, 2026).
- [8] Y.-K. Tsay, "Random Number Generation and Stream Ciphers." Accessed: Mar. 28, 2026. [Online]. Available: https://www.im.ntu.edu.tw/~tsay/dokuwiki/lib/exe/fetch.php?media=courses%3Ais2011%3Achapter7_random.pdf
- [9] "A5/1," *Wikipedia*, Oct. 16, 2021. <https://en.wikipedia.org/wiki/A5/1>
- [10] esraamahmoudhamza, "GitHub - esraamahmoudhamza/Steganography-GUI-with-Python," *GitHub*, 2025. <https://github.com/esraamahmoudhamza/Steganography-GUI-with-Python> (accessed Mar. 28, 2026).
- [11] M. Rianti, "PERBANDINGAN METODE STEGANOGRAFI DCT DAN DWT PADA BERKAS VIDEO MP4 SKRIPSI," Nov. 2021. Accessed: Mar. 28, 2026. [Online]. Available: <https://repository.uir.ac.id/10605/1/143510403.pdf>
- [12] Sandfreni, A. H. Azizah, and M. B. Ulum, "View of PERBANDINGAN TEKNIK STEGANOGRAFI DOMAIN SPASIAL DENGAN DOMAIN TRANSFORMASI PADA AUDIO," *Wicida.ac.id*, 2026. <https://jurnal.wicida.ac.id/index.php/sebatik/article/view/1642/567> (accessed Mar. 28, 2026).
- [13] A. S. Pratama and I. M. Suartana, "View of Analisis Kualitas Stego Video dalam Penyisipan Data Memanfaatkan Metode DCT-DWT," *Unesa.ac.id*, 2026. <https://journal.unesa.ac.id/index.php/jieet/article/view/13655/pdf> (accessed Mar. 28, 2026).
- [14] Yu Cheng, Zhenlin Luo, Zhaoxia Yin, Robust steganography with boundary-preserving overflow alleviation and adaptive error correction, *Expert Systems with Applications*, Volume 281, 2025, <https://www.sciencedirect.com/science/article/abs/pii/S0957417425012205>
- [15] M. F. Azhim, "Kompresi Citra Dokumen Rahasia Menggunakan Bit-Plane Slicing dan Huffman Coding dengan Selective Encryption," Institut Teknologi Bandung, Bandung, Indonesia., [https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/Makalah2025/Makalah-IF4073-PCD-2025%20\(11\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/Makalah2025/Makalah-IF4073-PCD-2025%20(11).pdf)

[16] X. Liu, B. Zhao, and W. Wang, "A robust video steganography algorithm for H.264/AVC based on secret sharing," Electronic Engineering College, Heilongjiang University, China, <https://dl.acm.org/doi/epdf/10.1145/3603781.3603938>

VI. Lampiran

Lampiran 1. Pranala repositori

<https://github.com/wiijaakk/II4021-Kriptografi-Tugas-II>

Lampiran 2. Pranala video demo

<bit.ly/VideoKriptoSendiWijakCia>

Lampiran 3. Pembagian Tugas

NIM	Nama	Pembagian Tugas
18223063	Sendi Putra Alicia	- Kode bagian implementasi inti steganografi video (merancang arsitektur <i>header</i>, operasi I/O video, menyusun logika penyisipan dan ekstraksi (Sequential/Random) untuk video <i>lossless</i> (AVI) - Mengimplementasi algoritma steganografi tahan kompresi (<i>robust</i>) khusus format MP4 - Bab 1 bagian Steganografi, Format Video AVI, Metode LSB (Least Significant Bit) - Bab 2 bagian Pemrosesan Video AVI, Metode Enkripsi & Dekripsi A5/1 yang Diimplementasikan, Metode LSB yang Diimplementasikan, Pemilihan Frame dan Urutan Piksel, Penyimpanan Metadata/Header, Pengecekan Kapasitas (Payload),
18223088	Wijaksana Aptaluhung	- Kode bagian implementasi algoritma enkripsi <i>stream cipher</i> A5/1, pemrosesan <i>file</i> atau teks untuk mengimplementasikan enkripsi, dan membuat algoritma <i>random embedding</i> lebih efisien - Bab 1 bagian Stream Cipher dan A5/1 - Bab 2 bagian Tech Stack dan Arsitektur Program, serta Metode Enkripsi & Dekripsi A5/1 yang

		Diimplementasikan - Bab 3 bagian Penyisipan dengan Berbagai Jenis Berkas - Bab 4 bagian Kesimpulan dan Saran
18223097	Audy Alicia Renatha Tirayoh	- Kode bagian GUI, Pengukuran Kualitas (MSE, PSNR, Histogram RGB), dan README - Bab 1 bagian Pengukuran Kualitas - Bab 2 bagian Pengukuran Kualitas (MSE, PSNR, dan Histogram) - Bab 3 bagian Semua Kombinasi Konfigurasi - Bab 3 bagian Penyisipan yang Melebihi Batas Kapasitas Sisip